

RL Notes

jasonxiong789

January 2025

1 Actor Critic

$$Q^\pi(s_t, a_t) = \sum_{t'=t}^T \mathbb{E}_\pi[r(s_{t'}, a_{t'}) | s_t, a_t]$$

$$V^\pi(s_t) = \sum_{t'=t}^T \mathbb{E}_\pi[r(s_{t'}, a_{t'}) | s_t] = \sum_{t'=t}^T \mathbb{E}_\pi[\mathbb{E}_\pi[r(s_{t'}, a_{t'}) | s_t, a_t] | s_t] = \mathbb{E}_\pi[Q^\pi(s_t, a_t) | s_t]$$

$$A^\pi(s_t, a_t) = Q^\pi(s_t, a_t) - V^\pi(s_t)$$

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=1}^T \nabla_\theta \log \pi_\theta(a_t | s_t) \sum_{t'=t}^T r(s_{t'}, a_{t'}) \right] \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^T \nabla_\theta \log \pi_\theta(a_{i,t} | s_{i,t}) A^\pi(s_{i,t}, a_{i,t})$$

Let $\tau_{a:b} = (s_a, a_a, s_{a+1}, a_{a+1}, \dots, s_b, a_b)$. We see the following hold:

$$\begin{aligned} \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t'=t}^T r(s_{t'}, a_{t'}) \right] &= \mathbb{E}_{\tau_{t:T} \sim \pi_\theta} \left[\mathbb{E}_{a_{t+1}, \tau_{t+2:T} \sim \pi_\theta} \left[\sum_{t'=t}^T r(s_{t'}, a_{t'}) \middle| s_t, a_t, s_{t+1} \right] \right] \\ &= \mathbb{E}_{\tau_{t:T} \sim \pi_\theta} \left[r(s_{t'}, a_{t'}) + \mathbb{E}_{a_{t+1}, \tau_{t+2:T} \sim \pi_\theta} \left[\sum_{t'=t+1}^T r(s_{t'}, a_{t'}) \middle| s_t, a_t, s_{t+1} \right] \right] \\ &= \mathbb{E}_{\tau_{t:T} \sim \pi_\theta} [r(s_{t'}, a_{t'}) + V^\pi(s_{t+1})] \\ \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t'=t}^T r(s_{t'}, a_{t'}) \right] &= \mathbb{E}_{\tau_{t:T} \sim \pi_\theta} \left[\mathbb{E}_{\tau_{t+1:T} \sim \pi_\theta} \left[\sum_{t'=t}^T r(s_{t'}, a_{t'}) \middle| s_t, a_t \right] \right] \\ &= \mathbb{E}_{\tau_{t:T} \sim \pi_\theta} [Q^\pi(s_t, a_t)] \end{aligned}$$

Thus, even though $\sum_{t'=t}^T r(s_{t'}, a_{t'})$, $r(s_{t'}, a_{t'}) + V^\pi(s_{t+1})$, and $Q^\pi(s_t, a_t)$ are all different RVs with increasing variance, their expected value is the same. Note, we can use any of the three inside the expected value in the expression for $J(\theta)$ since we conditioned on (s_t, a_t) when deriving $r(s_{t'}, a_{t'}) + V^\pi(s_{t+1})$ and $Q^\pi(s_t, a_t)$, which allows us to move $\nabla_\theta \log \pi_\theta(a_t | s_t)$ (an RV that is a function of (s_t, a_t)) outside the inner expectation.

Note that this also means when training a value function, we can use squared error on samples of any of the three (squared error converges to the expected value of the samples).

2 Variance

$$Var(\mathbb{E}[X|Y, Z]) = \mathbb{E}[\mathbb{E}[X|Y, Z]^2] - \mathbb{E}[\mathbb{E}[X|Y, Z]]^2 = \mathbb{E}[\mathbb{E}[X|Y, Z]^2] - \mathbb{E}[X]^2$$

$$Var(\mathbb{E}[X|Y]) = \mathbb{E}[\mathbb{E}[X|Y]^2] - \mathbb{E}[\mathbb{E}[X|Y]]^2 = \mathbb{E}[\mathbb{E}[X|Y]^2] - \mathbb{E}[X]^2$$

We want to prove the following:

$$Var(\mathbb{E}[X|Y, Z]) \geq Var(\mathbb{E}[X|Y]) \iff \mathbb{E}[\mathbb{E}[X|Y, Z]^2] \geq \mathbb{E}[\mathbb{E}[X|Y]^2]$$

Using the tower rule:

$$\mathbb{E}[X|Y] = \mathbb{E}[\mathbb{E}[X|Y, Z] | Y]$$

Since $f(x) = x^2$ is convex, Jensen's inequality ($\mathbb{E}[f(X)] \geq f(\mathbb{E}[X])$ for convex f) says

$$\begin{aligned} \mathbb{E}[\mathbb{E}[X|Y = y, Z]^2 | Y = y] &\geq \mathbb{E}[\mathbb{E}[X|Y = y, Z] | Y = y]^2 \\ \implies \mathbb{E}[\mathbb{E}[X|Y, Z]^2 | Y] &\geq \mathbb{E}[\mathbb{E}[X|Y, Z] | Y]^2 \\ &= \mathbb{E}[X|Y]^2 \end{aligned}$$

Taking the expected value of both sides, we get:

$$\mathbb{E}[\mathbb{E}[\mathbb{E}[X|Y, Z]^2 | Y]] \geq \mathbb{E}[\mathbb{E}[X|Y]^2]$$

Using the tower rule on the left, we get:

$$\mathbb{E}[\mathbb{E}[X|Y, Z]^2] \geq \mathbb{E}[\mathbb{E}[X|Y]^2] \iff Var(\mathbb{E}[X|Y, Z]) \geq Var(\mathbb{E}[X|Y])$$

Substitution $Z = X$, we also get that

$$Var(\mathbb{E}[X|Y, X]) = Var(X) \geq Var(\mathbb{E}[X|Y])$$

Note that

$$\begin{aligned} \nabla_{\theta} J(\theta) &= \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \sum_{t'=t}^T r(s_{t'}, a_{t'}) \right] \\ &= \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t=1}^T \mathbb{E}_{\tau_{t+1:T} \sim \pi_{\theta}} \left[\nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \sum_{t'=t}^T r(s_{t'}, a_{t'}) | s_t, a_t \right] \right] \\ &= \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \mathbb{E}_{\tau_{t+1:T} \sim \pi_{\theta}} \left[\sum_{t'=t}^T r(s_{t'}, a_{t'}) | s_t, a_t \right] \right] \\ &= \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) Q^{\pi}(s_t, a_t) \right] \\ &= \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\mathbb{E}_{\tau_{a_{t+1}, t+2:T} \sim \pi_{\theta}} \left[\sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \sum_{t'=t}^T r(s_{t'}, a_{t'}) | s_t, a_t, s_{t+1} \right] \right] \end{aligned}$$

Similarly,

$$\begin{aligned} \nabla_{\theta} J(\theta) &= \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \sum_{t'=t}^T r(s_{t'}, a_{t'}) \right] \\ &= \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \mathbb{E}_{a_{t+1}, \tau_{t+2:T} \sim \pi_{\theta}} \left[\sum_{t'=t}^T r(s_{t'}, a_{t'}) | s_t, a_t, s_{t+1} \right] \right] \\ &= \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \left(r(s_{t'}, a_{t'}) + \mathbb{E}_{a_{t+1}, \tau_{t+2:T} \sim \pi_{\theta}} \left[\sum_{t'=t+1}^T r(s_{t'}, a_{t'}) | s_t, a_t, s_{t+1} \right] \right) \right] \\ &= \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) (r(s_{t'}, a_{t'}) + V^{\pi}(s_{t+1})) \right] \end{aligned}$$

By the proof above, we also see that

$$\begin{aligned}
& Var \left(\sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \sum_{t'=t}^T r(s_{t'}, a_{t'}) \right) \\
& \geq Var \left(\sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) (r(s_{t'}, a_{t'}) + V^{\pi}(s_{t+1})) \right) \\
& \geq Var \left(\sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) Q^{\pi}(s_t, a_t) \right)
\end{aligned}$$

3 N-step Returns and GAE

The actor critic advantage estimator is as follows

$$\hat{A}_1^\pi(s_t, a_t) = r(s_t, a_t) + \gamma V_\phi^\pi(s_{t+1}) - V_\phi^\pi(s_t)$$

while the policy gradient advantage estimator is the following

$$\hat{A}_\infty^\pi(s_t, a_t) = \sum_{t'=t}^{\infty} \gamma^{t'-t} r(s_{t'}, a_{t'}) - V_\phi^\pi(s_t)$$

In the infinite time horizon setting, future rewards are discounted, and in the finite time horizon setting we are approximating fewer rewards with the biased value function estimator. For both cases, using the biased value function estimator in a future time step (with either smaller or fewer rewards) reduces the bias. However, continuing to use a single sample estimator farther into the future also increases variance since there are more states in the sample (trajectories branch out more farther into the future). Therefore, $\hat{A}_1$ has low variance and high bias while $\hat{A}_\infty$ has high variance and no bias. We can tradeoff bias and variance with something in between with an n-step return advantage estimator

$$\hat{A}_n^\pi(s_t, a_t) = \sum_{t'=t}^{t+n-1} \gamma^{t'-t} r(s_{t'}, a_{t'}) + \gamma^n V_\phi^\pi(s_{t+n+1}) - V_\phi^\pi(s_t)$$

where larger n has smaller bias and larger variance and smaller n has larger bias and lower variance. Instead of choosing a cutoff n , we can also use GAE (generalized advantage estimator) that averages all possible n step return advantage estimators.

$$\hat{A}_{GAE}^\pi(s_t, a_t) = (1 - \lambda) \sum_{n=1}^{\infty} \lambda^{n-1} \hat{A}_n^\pi(s_t, a_t)$$

If we let

$$\delta^V(s_t, a_t) = r(s_t, a_t) + \gamma V(s_{t+1}) - V(s_t) = -V(s_t) + r(s_t, a_t) + \gamma V(s_{t+1})$$

we can write

$$\hat{A}_n^\pi(s_t, a_t) = \sum_{l=t}^{t+n-1} \gamma^{l-t} \delta^V(s_l, a_l) = \sum_{l=0}^{n-1} \gamma^l \delta^V(s_{l+t}, a_{l+t})$$

Substituting this into the expression for $\hat{A}_{GAE}^\pi(s_t, a_t)$, we get

$$\begin{aligned}
\hat{A}_{GAE}^\pi(s_t, a_t) &= (1 - \lambda) \sum_{n=1}^{\infty} \lambda^{n-1} \hat{A}_n^\pi(s_t, a_t) \\
&= (1 - \lambda) \sum_{n=1}^{\infty} \lambda^{n-1} \sum_{l=0}^{n-1} \gamma^l \delta^V(s_{l+t}, a_{l+t}) \\
&= (1 - \lambda) \sum_{n=1}^{\infty} \sum_{l=0}^{n-1} \lambda^{n-1} \gamma^l \delta^V(s_{l+t}, a_{l+t}) \\
&= (1 - \lambda) \sum_{l=0}^{\infty} \sum_{n=l+1}^{\infty} \lambda^{n-1} \gamma^l \delta^V(s_{l+t}, a_{l+t}) \\
&= (1 - \lambda) \sum_{l=0}^{\infty} \frac{1}{1 - \lambda} (\lambda \gamma)^l \delta^V(s_{l+t}, a_{l+t}) \\
&= \sum_{l=0}^{\infty} (\lambda \gamma)^l \delta_{l+t}^V \\
&= \delta_t^V + \sum_{l=1}^{\infty} (\lambda \gamma)^l \delta_{l+t}^V \\
&= \delta_t^V + \lambda \gamma \sum_{l=1}^{\infty} (\lambda \gamma)^{l-1} \delta^V(s_{l+t}, a_{l+t}) \\
&= \delta_t^V + \lambda \gamma \sum_{l=0}^{\infty} (\lambda \gamma)^l \delta^V(s_{l+t+1}, a_{l+t+1}) \\
&= \delta_t^V + \lambda \gamma \hat{A}_{GAE}^\pi(s_{t+1}, a_{t+1})
\end{aligned}$$

Smaller λ puts smaller weight on rewards into the future, meaning lower variance and higher bias while larger λ has higher variance and lower bias. Note that an interesting observation is that γ impacts the bias and variance tradeoff the same as λ .

For the actual implementation, we can use the recursive definition or just compute δ_t^V for all time steps $1, \dots, T$ in each rollout, and for each state, set

$$\hat{A}_{GAE}^\pi(s_t, a_t) = \sum_{l=0}^{T-t} (\lambda \gamma)^l \delta_{l+t}^V$$

4 Learning Value Functions

Since

$$V^\pi(s_t) = \mathbb{E}_\pi \left[\sum_{t'=t} r(s_{t'}, a_{t'}) | s_t \right]$$

we want $V_\phi(s)$ to learn $\mathbb{E}_\pi [\sum_{t'=t} r(s_{t'}, a_{t'}) | s_t = s]$. Hence we give it samples of $\sum_{t'=t} r(s_{t'}, a_{t'})$ that have starting point $s_t = s$. Note that

$$\begin{aligned} \mathbb{E}_\pi \left[\sum_{t'=t} r(s_{t'}, a_{t'}) | s_t = s \right] &= \mathbb{E}_\pi \left[\mathbb{E}_\pi \left[\sum_{t'=t} r(s_{t'}, a_{t'}) \middle| s_t = s, a_t, s_{t+1} \right] \middle| s_t = s \right] \\ &= \mathbb{E}_\pi \left[r(s_t, a_t) + \mathbb{E}_\pi \left[\sum_{t'=t+1} r(s_{t'}, a_{t'}) \middle| s_t = s, a_t, s_{t+1} \right] \middle| s_t = s \right] \\ &= \mathbb{E}_\pi \left[r(s_t, a_t) + \mathbb{E}_\pi \left[\sum_{t'=t+1} r(s_{t'}, a_{t'}) \middle| s_{t+1} \right] \middle| s_t = s \right] \\ &= \mathbb{E}_\pi [r(s_t, a_t) + V^\pi(s_{t+1}) | s_t = s] \end{aligned}$$

Using $V_\phi(s)$ as a biased estimator for $V^\pi(s_{t+1})$, we can give samples of $r(s_t, a_t) + V^\pi(s_{t+1})$ that have starting point $s_t = s$ instead of $\sum_{t'=t} r(s_{t'}, a_{t'})$ and have $V_\phi(s)$ converge to the same expected value in theory (assuming $V_\phi(s)$ is unbiased). Similar logic can be applied for learning Q-Values.

5 Advanced Policy Gradients

First, we want to prove that

$$\mathbb{E} [\mathbb{E} [X|Y, Z]|Y] = \mathbb{E} [X|Y]$$

Starting with the definition of a conditional expectation:

$$\begin{aligned} \mathbb{E} [X|Y = z, Z = z] &= \int_x x f_{x|y,z}(x) dx \\ \mathbb{E} [\mathbb{E} [X|Y, Z]|Y] &= \int_z f_{z|y}(z) \int_x x f_{x|y,z}(x) dx dz \\ &= \int_z \int_x x f_{z|y}(z) f_{x|y,z}(x) dx dz \\ &= \int_z \int_x x f_{x,z|y}(x, z) dx dz \\ &= \mathbb{E} [X|Y] \end{aligned}$$

The following can be derived using the fact that we can switch the distribution over which the expected value is taken does not depend on the policy.

$$\begin{aligned} J(\theta) - J(\theta') &= \mathbb{E}_{\tau \sim p_{\theta'}(\tau)} \left[\sum_{t=0}^{\infty} \gamma^t (r(s_t, a_t) + \gamma V^{\pi_{\theta}}(s_{t+1}) - V^{\pi_{\theta}}(s_t)) \right] \\ &= \mathbb{E}_{\tau \sim p_{\theta'}(\tau)} \left[\sum_{t=0}^{\infty} \gamma^t (\mathbb{E}_{\tau \sim p_{\theta'}(\tau)} [r(s_t, a_t)|s_t, a_t] + \gamma \mathbb{E}_{\tau \sim p_{\theta'}(\tau)} [V^{\pi_{\theta}}(s_{t+1})|s_t, a_t] - V^{\pi_{\theta}}(s_t)) \right] \\ &= \mathbb{E}_{\tau \sim p_{\theta'}(\tau)} \left[\sum_{t=0}^{\infty} \gamma^t (\mathbb{E}_{\tau \sim p_{\theta}(\tau)} [r(s_t, a_t)|s_t, a_t] + \gamma \mathbb{E}_{s_{t+1} \sim p(s_{t+1}|s_t, a_t)} [V^{\pi_{\theta}}(s_{t+1})|s_t, a_t] - V^{\pi_{\theta}}(s_t)) \right] \\ &= \mathbb{E}_{\tau \sim p_{\theta'}(\tau)} \left[\sum_{t=0}^{\infty} \gamma^t (\mathbb{E}_{\tau \sim p_{\theta}(\tau)} [r(s_t, a_t)|s_t, a_t] + \gamma \mathbb{E}_{\tau \sim p_{\theta}(\tau)} [V^{\pi_{\theta}}(s_{t+1})|s_t, a_t] - V^{\pi_{\theta}}(s_t)) \right] \end{aligned}$$

Note in the last two steps, we were able to replace $p_{\theta'}(\tau)$ with $p_{\theta}(\tau)$ because all of the terms that depend on θ are inside the expectation and the distribution with respect to which the expectation is taken does not depend on θ .

Continuing, we get that

$$\begin{aligned}
& J(\theta') - J(\theta) \\
&= \mathbb{E}_{\tau \sim p_{\theta'}(\tau)} \left[\sum_{t=0}^{\infty} \gamma^t (r(s_t, a_t) + \gamma V^{\pi_{\theta}}(s_{t+1}) - V^{\pi_{\theta}}(s_t)) \right] \\
&= \mathbb{E}_{\tau \sim p_{\theta'}(\tau)} \left[\sum_{t=0}^{\infty} \gamma^t (\mathbb{E}_{\tau \sim p_{\theta}(\tau)} [r(s_t, a_t) | s_t, a_t] + \gamma \mathbb{E}_{\tau \sim p_{\theta}(\tau)} [V^{\pi_{\theta}}(s_{t+1}) | s_t, a_t] - V^{\pi_{\theta}}(s_t)) \right] \\
&= \mathbb{E}_{\tau \sim p_{\theta'}(\tau)} \left[\sum_{t=0}^{\infty} \gamma^t (\mathbb{E}_{\tau \sim p_{\theta}(\tau)} [r(s_t, a_t) + \gamma V^{\pi_{\theta}}(s_{t+1}) | s_t, a_t] - V^{\pi_{\theta}}(s_t)) \right] \\
&= \mathbb{E}_{\tau \sim p_{\theta'}(\tau)} \left[\sum_{t=0}^{\infty} \gamma^t \left(\mathbb{E}_{\tau \sim p_{\theta}(\tau)} \left[r(s_t, a_t) + \gamma \mathbb{E}_{\tau \sim p_{\theta}(\tau)} \left[\sum_{t'=t+1}^{\infty} \gamma^{t'-t-1} r(s_{t'}, a_{t'}) \middle| s_{t+1} \right] \middle| s_t, a_t \right] - V^{\pi_{\theta}}(s_t) \right) \right] \\
&= \mathbb{E}_{\tau \sim p_{\theta'}(\tau)} \left[\sum_{t=0}^{\infty} \gamma^t \left(\mathbb{E}_{\tau \sim p_{\theta}(\tau)} \left[r(s_t, a_t) + \gamma \mathbb{E}_{\tau \sim p_{\theta}(\tau)} \left[\sum_{t'=t+1}^{\infty} \gamma^{t'-t-1} r(s_{t'}, a_{t'}) \middle| s_t, a_t, s_{t+1} \right] \middle| s_t, a_t \right] - V^{\pi_{\theta}}(s_t) \right) \right] \\
&= \mathbb{E}_{\tau \sim p_{\theta'}(\tau)} \left[\sum_{t=0}^{\infty} \gamma^t \left(\mathbb{E}_{\tau \sim p_{\theta}(\tau)} \left[r(s_t, a_t) + \gamma \sum_{t'=t+1}^{\infty} \gamma^{t'-t-1} r(s_{t'}, a_{t'}) \middle| s_t, a_t \right] - V^{\pi_{\theta}}(s_t) \right) \right] \\
&= \mathbb{E}_{\tau \sim p_{\theta'}(\tau)} \left[\sum_{t=0}^{\infty} \gamma^t \left(\mathbb{E}_{\tau \sim p_{\theta}(\tau)} \left[r(s_t, a_t) + \sum_{t'=t+1}^{\infty} \gamma^{t'-t} r(s_{t'}, a_{t'}) \middle| s_t, a_t \right] - V^{\pi_{\theta}}(s_t) \right) \right] \\
&= \mathbb{E}_{\tau \sim p_{\theta'}(\tau)} \left[\sum_{t=0}^{\infty} \gamma^t \left(\mathbb{E}_{\tau \sim p_{\theta}(\tau)} \left[\sum_{t'=t}^{\infty} \gamma^{t'-t} r(s_{t'}, a_{t'}) \middle| s_t, a_t \right] - V^{\pi_{\theta}}(s_t) \right) \right] \\
&= \mathbb{E}_{\tau \sim p_{\theta'}(\tau)} \left[\sum_{t=0}^{\infty} \gamma^t (Q^{\pi_{\theta}}(s_t, a_t) - V^{\pi_{\theta}}(s_t)) \right] \\
&= \mathbb{E}_{\tau \sim p_{\theta'}(\tau)} \left[\sum_{t=0}^{\infty} \gamma^t A^{\pi_{\theta}}(s_t, a_t) \right]
\end{aligned}$$

where we got the 5th to last step using the Markov property and the 4th to last step using the lemma we proved initially.

Notice that

$$\begin{aligned}
J(\theta) - J(\theta') &= \mathbb{E}_{\tau \sim p_{\theta'}(\tau)} \left[\sum_{t=0}^{\infty} \gamma^t A^{\pi_{\theta}}(s_t, a_t) \right] \\
&= \sum_{t=0}^{\infty} \mathbb{E}_{s_t \sim p_{\theta'}(s_t)} [\mathbb{E}_{a_t \sim \pi_{\theta'}(a_t | s_t)} [\gamma^t A^{\pi_{\theta}}(s_t, a_t) | s_t]] \\
&= \sum_{t=0}^{\infty} \sum_s p_{\theta'}(s_t) \sum_a \pi_{\theta'}(a_t | s_t) \gamma^t A^{\pi_{\theta}}(s_t, a_t) \\
&= \sum_{t=0}^{\infty} \sum_s \gamma^t p_{\theta'}(s_t) \sum_a \pi_{\theta'}(a_t | s_t) A^{\pi_{\theta}}(s_t, a_t)
\end{aligned}$$

Since policy iteration picks the action with maximum Q value under theta for each state, we know that

$$\sum_a \pi_{\theta'}(a_t | s_t) A^{\pi_{\theta}}(s_t, a_t) = A^{\pi_{\theta}}(s_t, a_t^*)$$

where

$$a_t^* = \operatorname{argmax}_{a_t} Q^{\pi_{\theta}}(s_t, a_t)$$

Since

$$\begin{aligned}
A^{\pi_{\theta}}(s_t, a_t^*) &= Q^{\pi_{\theta}}(s_t, a_t^*) - V^{\pi_{\theta}}(s_t) \\
&= Q^{\pi_{\theta}}(s_t, a_t^*) - \mathbb{E}_{a_t \sim \pi_{\theta'}(a_t | s_t)} [Q^{\pi_{\theta}}(s_t, a_t)] \\
&\geq 0
\end{aligned}$$

we know that

$$\begin{aligned} J(\theta) - J(\theta') &= \sum_{t=0}^{\infty} \sum_s \gamma^t p_{\theta'}(s_t) \sum_a \pi_{\theta'}(a_t|s_t) A^{\pi_{\theta}}(s_t, a_t) \\ &\geq 0 \end{aligned}$$

6 Coupling

See <https://people.csail.mit.edu/costis/6896sp11/lec3s.pdf>, <https://maths.dur.ac.uk/stats/courses/StochProc34/1516Coupling34.pdf>

Essentially, coupling allows two random variables X and Y , with probability distributions $p_X(x)$ and $p_Y(y)$ respectively, that take on the same possible values to be defined on the same probability space as $\tilde{X}$ and $\tilde{Y}$ with a joint distribution $p_{\tilde{X}, \tilde{Y}}(\tilde{x}, \tilde{y})$ such that $p_X(x) = p_{\tilde{X}}(\tilde{x}) \forall x$ and $p_Y(y) = p_{\tilde{Y}}(\tilde{y}) \forall y$.

This lets us find relationships between $p_{\tilde{X}}(\tilde{x})$ and $p_{\tilde{Y}}(\tilde{y})$ that also hold for the original distributions $p_X(x)$ and $p_Y(y)$. For instance,

$$D_{TV}(p_X(x) || p_Y(y)) = D_{TV}(p_{\tilde{X}}(\tilde{x}) || p_{\tilde{Y}}(\tilde{y}))$$

and

$$\mathbb{E}_{p_X(x)} [f(x)] = \mathbb{E}_{p_{\tilde{X}}(\tilde{x})} [f(\tilde{x})]$$

Note this only lets us relate the distributions but not the random variables X and Y directly since they are on different probability spaces. For instance, the following cannot be said for all couplings:

$$\mathbb{E}_{p_X(x), p_Y(y)} [f(x, y)] = \mathbb{E}_{p_{\tilde{X}}, p_{\tilde{Y}}(\tilde{x})} [f(\tilde{x}, \tilde{y})]$$

In summary, since the original distributions and the distributions in the coupling are the same, anything that applies to the distributions in the coupling can also be applied to the original distributions.

The coupling lemma states that if

$$D_{TV}(p_\theta(s_t) || p_{\theta'}(s'_t)) = \epsilon$$

then there exists a coupling with two random variables $\tilde{s}_t$ and $\tilde{s}'_t$ with joint distributions $p_\theta(\tilde{s}_t)$ and $p_{\theta'}(\tilde{s}'_t)$ respectively such that

$$p(\tilde{s}_t = \tilde{s}'_t) = 1 - \epsilon$$

7 TRPO

Pinsker's inequality says that

$$D_{TV}(p(x)||q(x)) \leq \sqrt{\frac{1}{2}D_{KL}(p(x)||q(x))} \iff 2D_{TV}(p(x)||q(x))^2 \leq D_{KL}(p(x)||q(x)) \implies D_{TV}(p(x)||q(x))^2 \leq D_{KL}(p(x)||q(x))$$

$$\begin{aligned} J(\theta') - J(\theta) &= \mathbb{E}_{\tau \sim p_{\theta'}(\tau)} \left[\sum_{t=0}^{\infty} \gamma^t (r(s_t, a_t) + \gamma V^{\pi_{\theta'}}(s_{t+1}) - V^{\pi_{\theta}}(s_t)) \right] \\ &= \mathbb{E}_{\tau \sim p_{\theta'}(\tau)} \left[\sum_{t=0}^{\infty} \gamma^t A^{\pi_{\theta}}(s_t, a_t) \right] \end{aligned}$$

For all states s , let's say two policies $\pi_{\theta}(a|s)$ and $\pi_{\theta'}(a|s)$ satisfy

$$D_{TV}(\pi_{\theta}(a|s)||\pi_{\theta'}(a'|s))^2 \leq D_{KL}(\pi_{\theta}(a|s)||\pi_{\theta'}(a'|s)) \leq \epsilon^2 = \alpha \implies D_{TV}(\pi_{\theta}(a|s)||\pi_{\theta'}(a'|s)) \leq \epsilon = \sqrt{\alpha}$$

The coupling lemma tells us there exists a coupling $(\tilde{a}_t, \tilde{a}'_t) \forall t$ such that $P(\tilde{a}_t = \tilde{a}'_t | s_t) = 1 - \epsilon$. Similarly, the transition dynamic are the same regardless of policy, meaning

$$D_{TV}(p(s_t | s_{t-1}, a_{t-1}) || p(s'_t | s_{t-1}, a_{t-1})) = 0$$

Applying the coupling lemma again tells us there exists a coupling $(\tilde{s}_t, \tilde{s}'_t)$ such that $P(\tilde{s}_t = \tilde{s}'_t | s_{t-1}, a_{t-1}) = 1$. Combining the coupling of policies and transition probabilities, there exists a coupling of processes τ (trajectory under θ) and τ' (trajectory under θ') expressed as $(\tilde{\tau}, \tilde{\tau}')$ where we define a coupling of all states (using coupling $(\tilde{s}_t, \tilde{s}'_t)$) and actions (using coupling $(\tilde{a}_t, \tilde{a}'_t)$) such that the $\tilde{\tau} = \tilde{\tau}'$ until the first step that $\tilde{a}_t \neq \tilde{a}'_t$, after which the two processes/trajectories run independently. If we let $\tau: t$ represent $(s_0, a_1, s_1, \dots, a_t, s_t)$, the coupled processes satisfy

$$P(\tilde{\tau}_{:t} = \tilde{\tau}'_{:t}) = (1 - \epsilon)^t \iff P(\tilde{\tau}_{:t} \neq \tilde{\tau}'_{:t}) = 1 - (1 - \epsilon)^t$$

For any function of the state $f(s)$, we can express

$$\mathbb{E}_{\tilde{s}_t \sim p_{\theta}(\tilde{\tau})} [f(s_t)] = P(\tilde{\tau}_{:t} = \tilde{\tau}'_{:t}) \mathbb{E}_{\tilde{s}_t \sim p_{\theta}(\tilde{\tau})} [f(s_t) | \tilde{\tau}_{:t} = \tilde{\tau}'_{:t}] + P(\tilde{\tau}_{:t} \neq \tilde{\tau}'_{:t}) \mathbb{E}_{\tilde{s}_t \sim p_{\theta}(\tilde{\tau})} [f(s_t) | \tilde{\tau}_{:t} \neq \tilde{\tau}'_{:t}]$$

and

$$\mathbb{E}_{\tilde{s}'_t \sim p_{\theta'}(\tilde{\tau}')} [f(s'_t)] = P(\tilde{\tau}_{:t} = \tilde{\tau}'_{:t}) \mathbb{E}_{\tilde{s}'_t \sim p_{\theta'}(\tilde{\tau}')} [f(s'_t) | \tilde{\tau}_{:t} = \tilde{\tau}'_{:t}] + P(\tilde{\tau}_{:t} \neq \tilde{\tau}'_{:t}) \mathbb{E}_{\tilde{s}'_t \sim p_{\theta'}(\tilde{\tau}')} [f(s'_t) | \tilde{\tau}_{:t} \neq \tilde{\tau}'_{:t}]$$

Note that

$$\mathbb{E}_{\tilde{s}'_t \sim p_{\theta'}(\tilde{\tau}')} [f(s'_t) | \tilde{\tau}_{:t} = \tilde{\tau}'_{:t}] = \mathbb{E}_{\tilde{s}_t \sim p_{\theta}(\tilde{\tau})} [f(s_t) | \tilde{\tau}_{:t} = \tilde{\tau}'_{:t}]$$

Subtracting the two, we get

$$\begin{aligned} & \left| \mathbb{E}_{\tilde{s}'_t \sim p_{\theta'}(\tilde{\tau}')} [f(s'_t)] - \mathbb{E}_{\tilde{s}_t \sim p_{\theta}(\tilde{\tau})} [f(s_t)] \right| \\ &= \left| P(\tilde{\tau}_{:t} = \tilde{\tau}'_{:t}) \mathbb{E}_{\tilde{s}'_t \sim p_{\theta'}(\tilde{\tau}')} [f(s'_t) | \tilde{\tau}_{:t} = \tilde{\tau}'_{:t}] + P(\tilde{\tau}_{:t} \neq \tilde{\tau}'_{:t}) \mathbb{E}_{\tilde{s}'_t \sim p_{\theta'}(\tilde{\tau}')} [f(s'_t) | \tilde{\tau}_{:t} \neq \tilde{\tau}'_{:t}] \right. \\ &\quad \left. - P(\tilde{\tau}_{:t} = \tilde{\tau}'_{:t}) \mathbb{E}_{\tilde{s}_t \sim p_{\theta}(\tilde{\tau})} [f(s_t) | \tilde{\tau}_{:t} = \tilde{\tau}'_{:t}] + P(\tilde{\tau}_{:t} \neq \tilde{\tau}'_{:t}) \mathbb{E}_{\tilde{s}_t \sim p_{\theta}(\tilde{\tau})} [f(s_t) | \tilde{\tau}_{:t} \neq \tilde{\tau}'_{:t}] \right| \\ &= P(\tilde{\tau}_{:t} \neq \tilde{\tau}'_{:t}) \left| \mathbb{E}_{\tilde{s}'_t \sim p_{\theta'}(\tilde{\tau}')} [f(s'_t) | \tilde{\tau}_{:t} \neq \tilde{\tau}'_{:t}] - \mathbb{E}_{\tilde{s}_t \sim p_{\theta}(\tilde{\tau})} [f(s_t) | \tilde{\tau}_{:t} \neq \tilde{\tau}'_{:t}] \right| \\ &\leq (1 - (1 - \epsilon)^t) \left| \mathbb{E}_{\tilde{s}'_t \sim p_{\theta'}(\tilde{\tau}')} [f(s'_t) | \tilde{\tau}_{:t} \neq \tilde{\tau}'_{:t}] - \mathbb{E}_{\tilde{s}_t \sim p_{\theta}(\tilde{\tau})} [f(s_t) | \tilde{\tau}_{:t} \neq \tilde{\tau}'_{:t}] \right| \\ &\leq (1 - (1 - \epsilon)^t) \left(\max_s |f(s)| + \max_s |f(s)| \right) \\ &= 2(1 - (1 - \epsilon)^t) \max_s |f(s)| \end{aligned}$$

Since the distribution of τ and τ' are the same as the distribution of $\tilde{\tau}$ and $\tilde{\tau}'$ respectively, we know that

$$\begin{aligned} & \left| \mathbb{E}_{s_t \sim p_{\theta'}(\tau)} [f(s'_t)] - \mathbb{E}_{s_t \sim p_{\theta}(\tau)} [f(s_t)] \right| \leq 2(1 - (1 - \epsilon)^t) \max_s |f(s)| \\ & \implies \\ & \mathbb{E}_{s_t \sim p_{\theta'}(\tau)} [f(s'_t)] - \mathbb{E}_{s_t \sim p_{\theta}(\tau)} [f(s_t)] \geq -2(1 - (1 - \epsilon)^t) \max_s |f(s)| \\ & \iff \\ & \mathbb{E}_{s_t \sim p_{\theta'}(\tau)} [f(s'_t)] \geq \mathbb{E}_{s_t \sim p_{\theta}(\tau)} [f(s_t)] - 2(1 - (1 - \epsilon)^t) \max_s |f(s)| \end{aligned}$$

Thus, for policies $\pi_\theta(a|s)$ and $\pi_{\theta'}(a|s)$ that satisfy

$$D_{TV}(\pi_\theta(a|s)||\pi_{\theta'}(a'|s))^2 \leq D_{KL}(\pi_\theta(a|s)||\pi_{\theta'}(a'|s)) \leq \epsilon^2 \implies D_{TV}(\pi_\theta(a|s)||\pi_{\theta'}(a'|s)) \leq \epsilon$$

we get that

$$\begin{aligned} J(\theta') - J(\theta) &= \mathbb{E}_{\tau \sim p_{\theta'}(\tau)} \left[\sum_{t=0}^{\infty} \gamma^t A^{\pi_\theta}(s_t, a_t) \right] \\ &= \sum_{t=0}^{\infty} \gamma^t \mathbb{E}_{\tau \sim p_{\theta'}(\tau)} [A^{\pi_\theta}(s_t, a_t)] \\ &= \sum_{t=0}^{\infty} \gamma^t \mathbb{E}_{s_t \sim p_{\theta'}(s_t)} [\mathbb{E}_{a_t \sim \pi_{\theta'}(a_t|s_t)} [A^{\pi_\theta}(s_t, a_t)|s_t]] \\ &\geq \sum_{t=0}^{\infty} \gamma^t \mathbb{E}_{s_t \sim p_\theta(s_t)} [\mathbb{E}_{a_t \sim \pi_{\theta'}(a_t|s_t)} [A^{\pi_\theta}(s_t, a_t)|s_t]] - 2(1 - (1 - \epsilon)^t) \max_s |\mathbb{E}_{a_t \sim \pi_{\theta'}(a'_t|s)} [A^{\pi_\theta}(s, a_t)|s]| \\ &= \sum_{t=0}^{\infty} \gamma^t \mathbb{E}_{s_t \sim p_\theta(s_t)} [\mathbb{E}_{a_t \sim \pi_{\theta'}(a_t|s_t)} [A^{\pi_\theta}(s_t, a_t)|s_t]] - \sum_{t=0}^{\infty} \gamma^t 2(1 - (1 - \epsilon)^t) \max_s |\mathbb{E}_{a_t \sim \pi_{\theta'}(a'_t|s)} [A^{\pi_\theta}(s, a_t)|s]| \\ &= \sum_{t=0}^{\infty} \gamma^t \mathbb{E}_{s_t \sim p_\theta(s_t)} [\mathbb{E}_{a_t \sim \pi_{\theta'}(a_t|s_t)} [A^{\pi_\theta}(s_t, a_t)|s_t]] - 2 \max_s |\mathbb{E}_{a_t \sim \pi_{\theta'}(a'_t|s)} [A^{\pi_\theta}(s, a_t)|s]| \sum_{t=0}^{\infty} \gamma^t (1 - (1 - \epsilon)^t) \\ &= \sum_{t=0}^{\infty} \gamma^t \mathbb{E}_{s_t \sim p_\theta(s_t)} [\mathbb{E}_{a_t \sim \pi_{\theta'}(a_t|s_t)} [A^{\pi_\theta}(s_t, a_t)|s_t]] - 2 \max_s |\mathbb{E}_{a_t \sim \pi_{\theta'}(a'_t|s)} [A^{\pi_\theta}(s, a_t)|s]| \left(\frac{1}{1 - \gamma} - \frac{1}{1 - \gamma * (1 - \epsilon)} \right) \\ &= \sum_{t=0}^{\infty} \gamma^t \mathbb{E}_{s_t \sim p_\theta(s_t)} [\mathbb{E}_{a_t \sim \pi_{\theta'}(a_t|s_t)} [A^{\pi_\theta}(s_t, a_t)|s_t]] - 2 \max_s |\mathbb{E}_{a_t \sim \pi_{\theta'}(a'_t|s)} [A^{\pi_\theta}(s, a_t)|s]| \frac{1 - \gamma * (1 - \epsilon) - (1 - \gamma)}{(1 - \gamma)(1 - \gamma * (1 - \epsilon))} \\ &= \sum_{t=0}^{\infty} \gamma^t \mathbb{E}_{s_t \sim p_\theta(s_t)} [\mathbb{E}_{a_t \sim \pi_{\theta'}(a_t|s_t)} [A^{\pi_\theta}(s_t, a_t)|s_t]] - 2 \max_s |\mathbb{E}_{a_t \sim \pi_{\theta'}(a'_t|s)} [A^{\pi_\theta}(s, a_t)|s]| \frac{\gamma \epsilon}{(1 - \gamma)(1 - \gamma * (1 - \epsilon))} \\ &\geq \sum_{t=0}^{\infty} \gamma^t \mathbb{E}_{s_t \sim p_\theta(s_t)} [\mathbb{E}_{a_t \sim \pi_{\theta'}(a_t|s_t)} [A^{\pi_\theta}(s_t, a_t)|s_t]] - 2 \max_s |\mathbb{E}_{a_t \sim \pi_{\theta'}(a'_t|s)} [A^{\pi_\theta}(s, a_t)|s]| \frac{\gamma \epsilon}{(1 - \gamma)(1 - \gamma)} \\ &= \sum_{t=0}^{\infty} \gamma^t \mathbb{E}_{s_t \sim p_\theta(s_t)} [\mathbb{E}_{a_t \sim \pi_{\theta'}(a_t|s_t)} [A^{\pi_\theta}(s_t, a_t)|s_t]] - \frac{2\gamma \epsilon}{(1 - \gamma)^2} \max_s |\mathbb{E}_{a_t \sim \pi_{\theta'}(a'_t|s)} [A^{\pi_\theta}(s, a_t)|s]| \end{aligned}$$

Using the same coupling of policies from before

$$\begin{aligned} \mathbb{E}_{a_t \sim \pi_{\theta'}(a_t|s_t)} [A^{\pi_\theta}(s_t, a_t)] &= \mathbb{E}_{a'_t \sim \pi_{\theta'}(a'_t|s_t)} [A^{\pi_\theta}(s_t, a'_t)] \quad (\text{just changing variable notation}) \\ &= \mathbb{E}_{a'_t \sim \pi_{\theta'}(a'_t|s_t)} [A^{\pi_\theta}(s_t, a'_t)] - 0 \\ &= \mathbb{E}_{a'_t \sim \pi_{\theta'}(a'_t|s_t)} [A^{\pi_\theta}(s_t, a'_t)] - \mathbb{E}_{a_t \sim \pi_\theta(a_t|s_t)} [A^{\pi_\theta}(s_t, a_t)] \\ &= \mathbb{E}_{\tilde{a}'_t \sim \pi_{\theta'}(\tilde{a}'_t|s_t)} [A^{\pi_\theta}(s_t, \tilde{a}'_t)] - \mathbb{E}_{\tilde{a}_t \sim \pi_\theta(\tilde{a}_t|s_t)} [A^{\pi_\theta}(s_t, \tilde{a}_t)] \\ &= (P(\tilde{a}'_t = \tilde{a}_t|s_t) \mathbb{E}_{\tilde{a}'_t \sim \pi_{\theta'}(\tilde{a}'_t|s_t)} [A^{\pi_\theta}(s_t, \tilde{a}'_t)|\tilde{a}'_t = \tilde{a}_t|s_t] \\ &\quad + P(\tilde{a}'_t \neq \tilde{a}_t|s_t) \mathbb{E}_{\tilde{a}'_t \sim \pi_{\theta'}(\tilde{a}'_t|s_t)} [A^{\pi_\theta}(s_t, \tilde{a}'_t)|\tilde{a}'_t \neq \tilde{a}_t|s_t]) \\ &\quad - (P(\tilde{a}'_t = \tilde{a}_t|s_t) \mathbb{E}_{\tilde{a}_t \sim \pi_\theta(\tilde{a}_t|s_t)} [A^{\pi_\theta}(s_t, \tilde{a}_t)|\tilde{a}'_t = \tilde{a}_t] \\ &\quad + P(\tilde{a}'_t \neq \tilde{a}_t|s_t) \mathbb{E}_{\tilde{a}_t \sim \pi_\theta(\tilde{a}_t|s_t)} [A^{\pi_\theta}(s_t, \tilde{a}_t)|\tilde{a}'_t \neq \tilde{a}_t]) \\ &= P(\tilde{a}'_t \neq \tilde{a}_t|s_t) (\mathbb{E}_{\tilde{a}_t \sim \pi_\theta(\tilde{a}_t|s_t)} [A^{\pi_\theta}(s_t, \tilde{a}_t)|\tilde{a}'_t \neq \tilde{a}_t] - \mathbb{E}_{\tilde{a}_t \sim \pi_\theta(\tilde{a}_t|s_t)} [A^{\pi_\theta}(s_t, \tilde{a}_t)|\tilde{a}'_t \neq \tilde{a}_t]) \\ &= \epsilon (\mathbb{E}_{\tilde{a}_t \sim \pi_\theta(\tilde{a}_t|s_t)} [A^{\pi_\theta}(s_t, \tilde{a}_t)|\tilde{a}'_t \neq \tilde{a}_t] - \mathbb{E}_{\tilde{a}_t \sim \pi_\theta(\tilde{a}_t|s_t)} [A^{\pi_\theta}(s_t, \tilde{a}_t)|\tilde{a}'_t \neq \tilde{a}_t]) \\ &\leq \epsilon (|\mathbb{E}_{\tilde{a}_t \sim \pi_\theta(\tilde{a}_t|s_t)} [A^{\pi_\theta}(s_t, \tilde{a}_t)|\tilde{a}'_t \neq \tilde{a}_t]| + |\mathbb{E}_{\tilde{a}_t \sim \pi_\theta(\tilde{a}_t|s_t)} [A^{\pi_\theta}(s_t, \tilde{a}_t)|\tilde{a}'_t \neq \tilde{a}_t]|) \\ &\leq \epsilon \left(\max_{s,a} |A^{\pi_\theta}(s, a)| + \max_{s,a} |A^{\pi_\theta}(s, a)| \right) \\ &= 2\epsilon \max_{s,a} |A^{\pi_\theta}(s, a)| \end{aligned}$$

Plugging this into

$$\begin{aligned}
J(\theta') - J(\theta) &= \sum_{t=0}^{\infty} \gamma^t \mathbb{E}_{s_t \sim p_{\theta'}(s_t)} [\mathbb{E}_{a_t \sim \pi_{\theta'}(a_t|s_t)} [A^{\pi_{\theta}}(s_t, a_t)|s_t]] \\
&\geq \sum_{t=0}^{\infty} \gamma^t \mathbb{E}_{s_t \sim p_{\theta'}(s_t)} [\mathbb{E}_{a_t \sim \pi_{\theta'}(a_t|s_t)} [A^{\pi_{\theta}}(s_t, a_t)|s_t]] - \frac{2\gamma\epsilon}{(1-\gamma)^2} \max_s |\mathbb{E}_{a_t \sim \pi_{\theta'}(a_t|s)} [A^{\pi_{\theta}}(s, a_t)|s]| \\
&\geq \sum_{t=0}^{\infty} \gamma^t \mathbb{E}_{s_t \sim p_{\theta'}(s_t)} [\mathbb{E}_{a_t \sim \pi_{\theta'}(a_t|s_t)} [A^{\pi_{\theta}}(s_t, a_t)|s_t]] - \frac{2\gamma\epsilon}{(1-\gamma)^2} \max_s \left| 2\epsilon \max_{s,a} |A^{\pi_{\theta}}(s, a)| \right| \\
&= \sum_{t=0}^{\infty} \gamma^t \mathbb{E}_{s_t \sim p_{\theta'}(s_t)} [\mathbb{E}_{a_t \sim \pi_{\theta'}(a_t|s_t)} [A^{\pi_{\theta}}(s_t, a_t)|s_t]] - \frac{4\gamma\epsilon^2}{(1-\gamma)^2} \max_{s,a} |A^{\pi_{\theta}}(s, a)| \\
&= \sum_{t=0}^{\infty} \gamma^t \mathbb{E}_{s_t \sim p_{\theta'}(s_t)} [\mathbb{E}_{a_t \sim \pi_{\theta'}(a_t|s_t)} [A^{\pi_{\theta}}(s_t, a_t)|s_t]] - \frac{4\gamma\alpha}{(1-\gamma)^2} \max_{s,a} |A^{\pi_{\theta}}(s, a)|
\end{aligned}$$

Thus, maximizing the surrogate objective

$$\sum_{t=0}^{\infty} \gamma^t \mathbb{E}_{s_t \sim p_{\theta'}(s_t)} [\mathbb{E}_{a_t \sim \pi_{\theta'}(a_t|s_t)} [A^{\pi_{\theta}}(s_t, a_t)|s_t]]$$

maximizes the bound on the true objective, and the difference between the two scales linearly with α , the maximum KL divergence between policies $(\pi_{\theta}(a|s)$ and $\pi_{\theta'}(a'|s)$.

Note that the cs285 lecture just uses $\max_{s,a} |A^{\pi_{\theta}}(s, a)|$ instead of $2\epsilon \max_{s,a} |A^{\pi_{\theta}}(s, a)|$ as the upper bound of $\max_s |\mathbb{E}_{a_t \sim \pi_{\theta'}(a_t|s)} [A^{\pi_{\theta}}(s, a_t)|s]|$

We can also replace $A^{\pi_{\theta}}(s_t, a_t)$ with $Q^{\pi_{\theta}}(s_t, a_t)$ in the surrogate objective without changing the θ' that maximizes the surrogate objective:

$$\begin{aligned}
&\operatorname{argmax}_{\theta'} \sum_{t=0}^{\infty} \gamma^t \mathbb{E}_{s_t \sim p_{\theta'}(s_t)} [\mathbb{E}_{a_t \sim \pi_{\theta'}(a_t|s_t)} [A^{\pi_{\theta}}(s_t, a_t)|s_t]] \\
&= \operatorname{argmax}_{\theta'} \sum_{t=0}^{\infty} \gamma^t \mathbb{E}_{s_t \sim p_{\theta'}(s_t)} [\mathbb{E}_{a_t \sim \pi_{\theta'}(a_t|s_t)} [Q^{\pi_{\theta}}(s_t, a_t) - V^{\pi_{\theta}}(s_t)|s_t]] \\
&= \operatorname{argmax}_{\theta'} \sum_{t=0}^{\infty} (\gamma^t \mathbb{E}_{s_t \sim p_{\theta'}(s_t)} [\mathbb{E}_{a_t \sim \pi_{\theta'}(a_t|s_t)} [Q^{\pi_{\theta}}(s_t, a_t)|s_t]] - \gamma^t \mathbb{E}_{s_t \sim p_{\theta'}(s_t)} [\mathbb{E}_{a_t \sim \pi_{\theta'}(a_t|s_t)} [V^{\pi_{\theta}}(s_t)|s_t]]) \\
&= \operatorname{argmax}_{\theta'} \sum_{t=0}^{\infty} (\gamma^t \mathbb{E}_{s_t \sim p_{\theta'}(s_t)} [\mathbb{E}_{a_t \sim \pi_{\theta'}(a_t|s_t)} [Q^{\pi_{\theta}}(s_t, a_t)|s_t]] - \gamma^t \mathbb{E}_{s_t \sim p_{\theta'}(s_t)} [V^{\pi_{\theta}}(s_t)]) \\
&= \operatorname{argmax}_{\theta'} \sum_{t=0}^{\infty} \gamma^t \mathbb{E}_{s_t \sim p_{\theta'}(s_t)} [\mathbb{E}_{a_t \sim \pi_{\theta'}(a_t|s_t)} [Q^{\pi_{\theta}}(s_t, a_t)|s_t]] - \sum_{t=0}^{\infty} \gamma^t \mathbb{E}_{s_t \sim p_{\theta'}(s_t)} [V^{\pi_{\theta}}(s_t)] \\
&= \operatorname{argmax}_{\theta'} \sum_{t=0}^{\infty} \gamma^t \mathbb{E}_{s_t \sim p_{\theta'}(s_t)} [\mathbb{E}_{a_t \sim \pi_{\theta'}(a_t|s_t)} [Q^{\pi_{\theta}}(s_t, a_t)|s_t]] \quad (\text{since the second term does not depend on } \theta')
\end{aligned}$$

We can also apply importance sampling on the inner expectation of the surrogate objective using any arbitrary distribution over actions, which is typically conditioned on the state. Calling this arbitrary distribution as $q(a_t|s_t)$, we get

$$\sum_{t=0}^{\infty} \gamma^t \mathbb{E}_{s_t \sim p_{\theta'}(s_t)} [\mathbb{E}_{a_t \sim \pi_{\theta'}(a_t|s_t)} [Q^{\pi_{\theta}}(s_t, a_t)|s_t]] = \sum_{t=0}^{\infty} \gamma^t \mathbb{E}_{s_t \sim p_{\theta'}(s_t)} \left[\mathbb{E}_{a_t \sim q(a_t|s_t)} \left[\frac{\pi_{\theta'}(a_t|s_t)}{q(a_t|s_t)} Q^{\pi_{\theta}}(s_t, a_t) \middle| s_t \right] \right]$$

The TRPO algorithm does the following:

1. Estimate the surrogate objective using "single path" or "vine" estimation.

"single path"

- Sets $q(a_t|s_t) = \pi_{\theta}(a_t|s_t)$

- Runs the old policy $\pi_\theta(a_t|s_t)$ to gather R sample rollouts
- For each encountered state action pair (s_t, a_t) pair at time t from trajectory r , we approximate $Q^{\pi_\theta}(s_t, a_t)$ using as the discounted sum of future rewards along the rollout that (s_t, a_t) was encountered, which we can denote as $\hat{Q}^{\pi_\theta}(s_{r,t}, a_{r,t})$. This is essentially just a single sample estimate and is just the discounted reward to go. We then estimate the surrogate objective as $\frac{1}{RT} \sum_{r=1}^R \sum_{t=1}^T \frac{\pi_{\theta'}(s_{r,t}|a_{r,t})}{q(s_{r,t}|a_{r,t})} \hat{Q}^{\pi_\theta}(s_{r,t}, a_{r,t})$ (e.g. take an average across all seen (s_t, a_t) pairs).

”vine”

- Sets $q(a_t|s_t) = \pi_\theta(a_t|s_t)$ for continuous action spaces $\mathcal{A}$ and $q(a_t|s_t) = U(0, |\mathcal{A}| - 1)$ (uniform distribution across all actions) for discrete action spaces $\mathcal{A}$.
- Runs the old policy $\pi_\theta(a_t|s_t)$ to gather R sample rollouts
- We sample N states $s_1, \dots, s_N$ from the sample rollouts. For each sampled state s_n , we sample K actions $a_{n,1}, \dots, a_{n,K}$ from $q(a_t|s_n)$ and then complete a rollout with policy $\pi_\theta(a_t|s_t)$ for each $a_{n,k}$. Note for small discrete action spaces, we can just set $K = |\mathcal{A}|$ and complete a rollout for each action. This essentially makes the expected value over q exact (note the Q^π inside the expected value is still an estimate). We approximate $Q^{\pi_\theta}(s_t, a_t)$ for each of the K rollouts using the discounted sum of future rewards along the rollout, which we can denote as $\hat{Q}^{\pi_\theta}(s_n, a_{n,k})$. We then estimate the surrogate objective as $\frac{1}{NK} \sum_{n=1}^N \sum_{k=1}^K \frac{\pi_{\theta'}(a_{n,k}|s_n)}{q(a_{n,k}|s_n)} \hat{Q}^{\pi_\theta}(s_n, a_{n,k})$.

The TRPO paper also uses the self normalized estimator for continuous spaces: $\frac{1}{N} \sum_{n=1}^N \frac{\sum_{k=1}^K \frac{1}{K} \frac{\pi_{\theta'}(a_{n,k}|s_n)}{q(a_{n,k}|s_n)} \hat{Q}^{\pi_\theta}(s_n, a_{n,k})}{\frac{1}{K} \sum_{k=1}^K \frac{\pi_{\theta'}(a_{n,k}|s_n)}{q(a_{n,k}|s_n)}}$.

Note that we could also use GAE or actor critic here rather than just using an estimate of the return.

2. Run Conjugate Gradient to Get the Directions the Parameters Should Move

As with Natural gradients, we want to move the parameters in the direction of $F^{-1} \nabla_{\theta'} J(\theta')$, where

$$F = \nabla_{\theta'}^2 D_{KL}(\pi_\theta(a|s) || \pi_{\theta'}(a|s))$$

The bar on top of the KL divergence term just means we are taking an expected value across sampled states.

$$D_{KL}(\pi_\theta(a|s) || \pi_{\theta'}(a|s)) = d_{kl}(\mu(\theta'), \mu(\theta))$$

Where d_{kl} is a function that takes in two vectors $\mu(\theta')$ and $\mu(\theta)$ of length m that parameterize $\pi_\theta(a|s)$ and $\pi_{\theta'}(a|s)$ respectively and computes the kl divergence. Essentially, $\mu(\theta')$ and $\mu(\theta)$ are the outputs of the policy networks (e.g. their final layer) $\pi_\theta(a|s)$ and $\pi_{\theta'}(a|s)$ for input s . For instance, in continuous spaces if we represent the policy as a Gaussian with diagonal covariance, $\mu(\theta)$ can be a vector that is a concatenation of a vector that is the mean of a Gaussian and another vector that is the log of the square root (e.g. log standard deviation) of the covariance matrix diagonal.

When θ has n parameters, we can express the Hessian of the KL divergence as follows using the chain rule

$$\begin{aligned} \nabla_{\theta'} d_{kl}(\mu(\theta), \mu(\theta')) &= \begin{bmatrix} \sum_{a=1}^m = \frac{\partial}{\partial \mu(\theta')_a} d_{kl}(\mu(\theta'), \mu(\theta)) \frac{\partial \mu(\theta')_a}{\partial \theta_1} \\ \vdots \\ \sum_{a=1}^m = \frac{\partial}{\partial \mu(\theta')_a} d_{kl}(\mu(\theta'), \mu(\theta)) \frac{\partial \mu(\theta')_a}{\partial \theta_n} \end{bmatrix} \\ \implies \nabla_{\theta'}^2 d_{kl}(\mu(\theta), \mu(\theta')) &= \nabla_{\theta'} \begin{bmatrix} \sum_{a=1}^m \left(\frac{\partial}{\partial \mu(\theta')_a} d_{kl}(\mu(\theta'), \mu(\theta)) \right) \frac{\partial \mu(\theta')_a}{\partial \theta_1} \\ \vdots \\ \sum_{a=1}^m \left(\frac{\partial}{\partial \mu(\theta')_a} d_{kl}(\mu(\theta'), \mu(\theta)) \right) \frac{\partial \mu(\theta')_a}{\partial \theta_n} \end{bmatrix} \\ &= F_s \end{aligned}$$

where

$$\begin{aligned}
F_{s_{i,j}} &= \sum_{a=1}^m \frac{\partial}{\partial \theta'_i} \left(\left(\frac{\partial}{\partial \mu(\theta')_a} d_{kl}(\mu(\theta'), \mu(\theta)) \right) \frac{\partial \mu(\theta')_a}{\partial \theta_j} \right) \\
&= \sum_{a=1}^m \left(\frac{\partial}{\partial \theta'_i} \left(\frac{\partial}{\partial \mu(\theta')_a} d_{kl}(\mu(\theta'), \mu(\theta)) \right) \right) * \frac{\partial \mu(\theta')_a}{\partial \theta_j} + \left(\frac{\partial}{\partial \mu(\theta')_a} d_{kl}(\mu(\theta'), \mu(\theta)) \right) * \frac{\partial^2 \mu(\theta')_a}{\partial \theta_i \partial \theta_j} \\
&= \sum_{a=1}^m \left(\frac{\partial}{\partial \theta'_i} \left(\frac{\partial}{\partial \mu(\theta')_a} d_{kl}(\mu(\theta'), \mu(\theta)) \right) \right) * \frac{\partial \mu(\theta')_a}{\partial \theta_j} + 0 * \frac{\partial^2 \mu(\theta')_a}{\partial \theta_i \partial \theta_j} \quad (1) \\
&= \sum_{a=1}^m \sum_{b=1}^m \left(\left(\frac{\partial}{\partial \mu(\theta')_b} \left(\frac{\partial}{\partial \mu(\theta')_a} d_{kl}(\mu(\theta'), \mu(\theta)) \right) \right) \frac{\partial \mu(\theta')_b}{\partial \theta_i} \right) * \frac{\partial \mu(\theta')_a}{\partial \theta_j} \\
&= \sum_{a=1}^m \sum_{b=1}^m \left(\frac{\partial}{\partial \mu(\theta')_b} \frac{\partial}{\partial \mu(\theta')_a} d_{kl}(\mu(\theta'), \mu(\theta)) \right) \frac{\partial \mu(\theta')_b}{\partial \theta_i} \frac{\partial \mu(\theta')_a}{\partial \theta_j} \\
&= \sum_{a=1}^m \sum_{b=1}^m \frac{\partial \mu(\theta')_b}{\partial \theta_i} \frac{\partial \mu(\theta')_a}{\partial \theta_j} \left(\frac{\partial^2}{\partial \mu(\theta')_b \partial \mu(\theta')_a} d_{kl}(\mu(\theta'), \mu(\theta)) \right) \\
&= \sum_{a=1}^m \sum_{b=1}^m \frac{\partial \mu(\theta')_a}{\partial \theta_i} \frac{\partial \mu(\theta')_b}{\partial \theta_j} \left(\frac{\partial^2}{\partial \mu(\theta')_a \partial \mu(\theta')_b} d_{kl}(\mu(\theta'), \mu(\theta)) \right) \quad (\text{Switch summation order and a,b variable names})
\end{aligned}$$

where we use the fact that the gradient of the KL divergence with respect to the parameters of the underlying distribution is $\vec{0}$, meaning the *ath* element is also 0 (we can treat the model parameters are just one layer here). Thus, for a particular sampled state s we can express F_s as

$$F_s = J^T M J$$

where

$$\begin{aligned}
J_{i,j} &= \frac{\partial \mu(\theta')_i}{\partial \theta_j} \\
M_{i,j} &= \frac{\partial^2}{\partial \mu(\theta')_i \partial \mu(\theta')_j} d_{kl}(\mu(\theta'), \mu(\theta))
\end{aligned}$$

Since it is difficult to enforce the KL constrain across all states, we just constrain D_{KL}^- and $\bar{d}_{kl}$. Note that the actual implementation does not do this factorization and just uses the fact that the Hessian vector product can be expressed as

$$F = \nabla_{\theta'}^2 D_{KL}^-(\pi_{\theta}(a|s) || \pi_{\theta'}(a|s)) x = \nabla_{\theta'} \left((\nabla_{\theta'} D_{KL}^-(\pi_{\theta}(a|s) || \pi_{\theta'}(a|s))^T x \right)$$

Using this Hessian vector product, we run conjugate gradient algorithm to solve for the search direction s

$$\begin{aligned}
F_s &= \nabla_{\theta'} J(\theta') \\
\implies s &\approx F^{-1} J(\theta')
\end{aligned}$$

To calculate the approximate step size β , we first solve for a reasonable step size using the second order approximation of $D_{KL}^-(\pi_{\theta}(a|s) || \pi_{\theta'}(a|s))$ around $\theta' = \theta$:

$$\begin{aligned}
D_{KL}^- &\approx \frac{1}{2} ((\theta + \beta s) - (\theta))^T F ((\theta + \beta s) - (\theta)) = \delta \\
\implies \frac{1}{2} \beta^2 s^T F s &= \delta \\
\implies \beta &= \sqrt{\frac{2\delta}{s^T F s}}
\end{aligned}$$

For some $\alpha \in (0, 1)$, we use $\alpha^i \beta$ as the step size, where i is the smallest integer such that the average KL constraint is satisfied and the surrogate objective improves (this is line search).

8 PPO

For all states s , let's say two policies $\pi_\theta(a|s)$ and $\pi_{\theta'}(a|s)$ satisfy

$$D_{TV}(\pi_\theta(a|s) || \pi_{\theta'}(a'|s))^2 \leq D_{KL}(\pi_\theta(a|s) || \pi_{\theta'}(a'|s)) \leq \epsilon^2 = \alpha \implies D_{TV}(\pi_\theta(a|s) || \pi_{\theta'}(a'|s)) \leq \epsilon = \sqrt{\alpha}$$

Then

$$\begin{aligned} J(\theta') - J(\theta) &= \mathbb{E}_{\tau \sim p_{\theta'}(\tau)} \left[\sum_{t=0}^{\infty} \gamma^t A^{\pi_\theta}(s_t, a_t) \right] \\ &= \sum_{t=0}^{\infty} \gamma^t \mathbb{E}_{\tau \sim p_{\theta'}(\tau)} [A^{\pi_\theta}(s_t, a_t)] \\ &\geq \sum_{t=0}^{\infty} \gamma^t \mathbb{E}_{s_t \sim p_\theta(s_t)} [\mathbb{E}_{a_t \sim \pi_{\theta'}(a_t|s_t)} [A^{\pi_\theta}(s_t, a_t) | s_t]] - \frac{2\gamma\epsilon}{(1-\gamma)^2} \max_s \left| 2\epsilon \max_{s,a} |A^{\pi_\theta}(s, a)| \right| \\ &\geq \sum_{t=0}^{\infty} \gamma^t \mathbb{E}_{s_t \sim p_\theta(s_t)} \left[\mathbb{E}_{a_t \sim q(a_t)} \left[\frac{\pi_{\theta'}(a_t|s_t)}{q(a_t)} A^{\pi_\theta}(s_t, a_t) \middle| s_t \right] \right] - \frac{2\gamma\epsilon}{(1-\gamma)^2} \max_s \left| 2\epsilon \max_{s,a} |A^{\pi_\theta}(s, a)| \right| \\ &= \sum_{t=0}^{\infty} \gamma^t \mathbb{E}_{s_t \sim p_\theta(s_t)} \left[\mathbb{E}_{a_t \sim q(a_t)} \left[\frac{\pi_{\theta'}(a_t|s_t)}{q(a_t)} A^{\pi_\theta}(s_t, a_t) \middle| s_t \right] \right] - \frac{4\gamma\epsilon^2}{(1-\gamma)^2} \max_{s,a} |A^{\pi_\theta}(s, a)| \\ &= \sum_{t=0}^{\infty} \gamma^t \mathbb{E}_{s_t \sim p_\theta(s_t)} \left[\mathbb{E}_{a_t \sim q(a_t)} \left[\frac{\pi_{\theta'}(a_t|s_t)}{q(a_t)} A^{\pi_\theta}(s_t, a_t) \middle| s_t \right] \right] - \frac{4\gamma\alpha}{(1-\gamma)^2} \max_{s,a} |A^{\pi_\theta}(s, a)| \end{aligned}$$

See TRPO section for full derivation.

Setting $q(a_t) = \pi_\theta(a_t|s_t)$ (e.g the policy we used to collect data, which may not be the most recent policy), we get

$$\begin{aligned} J(\theta') - J(\theta) &\geq \sum_{t=0}^{\infty} \gamma^t \mathbb{E}_{s_t \sim p_\theta(s_t)} \left[\mathbb{E}_{a_t \sim \pi_\theta(a_t|s_t)} \left[\frac{\pi_{\theta'}(a_t|s_t)}{\pi_\theta(a_t|s_t)} A^{\pi_\theta}(s_t, a_t) \middle| s_t \right] \right] - \frac{4\gamma\alpha}{(1-\gamma)^2} \max_{s,a} |A^{\pi_\theta}(s, a)| \\ &= \sum_{t=0}^{\infty} \gamma^t \mathbb{E}_{\tau \sim p_\theta(\tau)} \left[\frac{\pi_{\theta'}(a_t|s_t)}{\pi_\theta(a_t|s_t)} A^{\pi_\theta}(s_t, a_t) \right] - \frac{4\gamma\alpha}{(1-\gamma)^2} \max_{s,a} |A^{\pi_\theta}(s, a)| \end{aligned}$$

Thus, when the KL divergence between the new and old policy is small, maximizing the surrogate objective with respect to θ'

$$\sum_{t=0}^{\infty} \gamma^t \mathbb{E}_{\tau \sim p_\theta(\tau)} \left[\frac{\pi_{\theta'}(a_t|s_t)}{\pi_\theta(a_t|s_t)} A^{\pi_\theta}(s_t, a_t) \right]$$

also maximizes the original objective $J(\theta') - J(\theta)$. TRPO approximated the KL divergence constraint using the Hessian of the KL divergence (second order approximation) and did not enforce the constraint over all states, but instead enforced the approximated constraint for the average of the KL divergences across states.

To simplify the constraint, PPO tries two methods, one of which is taking the minimum advantage between the surrogate objective and a surrogate object that clips $\frac{\pi_{\theta'}(a_t|s_t)}{\pi_\theta(a_t|s_t)}$ within ϵ of 1:

$$J(\theta') - J(\theta) = \sum_{t=0}^{\infty} \gamma^t \mathbb{E}_{\tau \sim p_\theta(\tau)} \left[\min \left(\frac{\pi_{\theta'}(a_t|s_t)}{\pi_\theta(a_t|s_t)} A^{\pi_\theta}(s_t, a_t), \text{clip} \left(\frac{\pi_{\theta'}(a_t|s_t)}{\pi_\theta(a_t|s_t)}, 1 - \epsilon, 1 + \epsilon \right) A^{\pi_\theta}(s_t, a_t) \right) \right]$$

This disincentivizes the policy from shifting too far from the old policy since any state action pair that has $\frac{\pi_{\theta'}(a_t|s_t)}{\pi_\theta(a_t|s_t)}$ outside of ϵ of 1 cannot improve the objective further by moving further from 1. Moreover, taking the minimum of the unclipped and clipped objective means that being outside of the clipping interval can only decrease the objective, not allowing the model to take advantage of the clipping by potentially blowing up (or down) the ratio for a state with clipping to improve the reward at a different time step. Note that the clipped objective is also the same as

$$\text{clip} \left(\frac{\pi_{\theta'}(a_t|s_t)}{\pi_\theta(a_t|s_t)}, 1 - \epsilon, 1 + \epsilon \right) A^{\pi_\theta}(s_t, a_t) = \begin{cases} (1 + \epsilon) A^{\pi_\theta}(s_t, a_t) & A^{\pi_\theta}(s_t, a_t) \geq 0 \\ (1 - \epsilon) A^{\pi_\theta}(s_t, a_t) & A^{\pi_\theta}(s_t, a_t) < 0. \end{cases}$$

The other method for PPO is adding a penalty for the KL divergence that is weighted by an adaptive coefficient β

$$J(\theta') - J(\theta) = \sum_{t=0}^{\infty} \gamma^t \mathbb{E}_{\tau \sim p_{\theta}(\tau)} \left[\frac{\pi_{\theta'}(a_t | s_t)}{\pi_{\theta}(a_t | s_t)} A^{\pi_{\theta}}(s_t, a_t) - \beta \text{KL}[\pi_{\theta}(\cdot | s_t), \pi_{\theta'}(\cdot | s_t)] \right]$$

where β is between data collections based on a d_{targ} (target KL divergence):

$$\beta = \begin{cases} \beta/2, & \text{If } \mathbb{E}_{\tau \sim p_{\theta}(\tau)} [\text{KL}[\pi_{\theta}(\cdot | s_t), \pi_{\theta'}(\cdot | s_t)]] < d_{targ}/1.5 \\ \beta \times 2, & \text{If } \mathbb{E}_{\tau \sim p_{\theta}(\tau)} [\text{KL}[\pi_{\theta}(\cdot | s_t), \pi_{\theta'}(\cdot | s_t)]] > d_{targ} * 1.5 \end{cases}$$

In the paper, they initialized β to 1 for the adaptive penalty with d_{targ} values as (0.003, 0.01, 0.03).

Essentially, if the KL constraint was being violated too much, we increase the KL divergence penalty and vice versa since a fixed penalty coefficient is hard to manually tune.

9 Off Policy Policy Gradient Baseline

From before, we know the off policy policy gradient looks like:

$$\begin{aligned}
\nabla_{\theta'} J(\theta') &= \nabla_{\theta'} \mathbb{E}_{\tau \sim p_{\theta}(\tau)} \left[\sum_{t'=1}^T r(s_{t'}, a_{t'}) \right] \\
&= \nabla_{\theta'} \mathbb{E}_{\tau \sim p_{\theta}(\tau)} \left[\frac{p_{\theta'}(\tau)}{p_{\theta}(\tau)} \sum_{t'=1}^T r(s_{t'}, a_{t'}) \right] \\
&= \mathbb{E}_{\tau_{1:T} \sim p_{\theta}(\tau)} \left[\sum_{t=1}^T \nabla_{\theta'} \log(\pi_{\theta'}(a_t | s_t)) \left(\prod_{t''=1}^t \frac{\pi_{\theta'}(a_{t''} | s_{t''})}{\pi_{\theta}(a_{t''} | s_{t''})} \right) \left(\sum_{t'=t}^T r(s_{t'}, a_{t'}) \left(\prod_{t''=t+1}^{t'} \frac{\pi_{\theta'}(a_{t''} | s_{t''})}{\pi_{\theta}(a_{t''} | s_{t''})} \right) \right) \right]
\end{aligned}$$

Just like with the original policy gradient, we can add a baseline that is a function of s_t to the reward to go since for any t and t' :

$$\begin{aligned}
&\mathbb{E}_{\tau_{1:T} \sim p_{\theta}(\tau)} \left[\nabla_{\theta'} \log(\pi_{\theta'}(a_t | s_t)) \left(\prod_{t''=1}^{t'} \frac{\pi_{\theta'}(a_{t''} | s_{t''})}{\pi_{\theta}(a_{t''} | s_{t''})} \right) f(s_t) \right] \\
&= \mathbb{E}_{\tau_{1:t-1}, s_t \sim p_{\theta}(\tau)} \left[\mathbb{E}_{a_t, \tau_{t+1:T} \sim p_{\theta}(\tau)} \left[\nabla_{\theta'} \log(\pi_{\theta'}(a_t | s_t)) \left(\prod_{t''=1}^{t'} \frac{\pi_{\theta'}(a_{t''} | s_{t''})}{\pi_{\theta}(a_{t''} | s_{t''})} \right) f(s_t) \right] \middle| \tau_{1:t-1}, s_t \right] \\
&= \mathbb{E}_{\tau_{1:t-1}, s_t \sim p_{\theta}(\tau)} \left[\left(\prod_{t''=1}^{t-1} \frac{\pi_{\theta'}(a_{t''} | s_{t''})}{\pi_{\theta}(a_{t''} | s_{t''})} \right) f(s_t) \mathbb{E}_{a_t, \tau_{t+1:T} \sim p_{\theta}(\tau)} \left[\nabla_{\theta'} \log(\pi_{\theta'}(a_t | s_t)) \left(\prod_{t''=t}^{t'} \frac{\pi_{\theta'}(a_{t''} | s_{t''})}{\pi_{\theta}(a_{t''} | s_{t''})} \right) \right] \middle| \tau_{1:t-1}, s_t \right]
\end{aligned}$$

Looking at the inner expectation, we see that

$$\begin{aligned}
&\mathbb{E}_{a_t, \tau_{t+1:T} \sim p_{\theta}(\tau)} \left[\nabla_{\theta'} \log(\pi_{\theta'}(a_t | s_t)) \left(\prod_{t''=t}^{t'} \frac{\pi_{\theta'}(a_{t''} | s_{t''})}{\pi_{\theta}(a_{t''} | s_{t''})} \right) \right] \middle| \tau_{1:t-1}, s_t \\
&= \mathbb{E}_{a_t, \tau_{t+1:T} \sim p_{\theta}(\tau)} \left[\mathbb{E}_{a_{t'} \sim p_{\theta}(\tau)} \left[\nabla_{\theta'} \log(\pi_{\theta'}(a_t | s_t)) \left(\prod_{t''=t}^{t'} \frac{\pi_{\theta'}(a_{t''} | s_{t''})}{\pi_{\theta}(a_{t''} | s_{t''})} \right) \right] \middle| \tau_{1:t-1}, s_{t'} \right] \middle| \tau_{1:t-1}, s_t \\
&= \mathbb{E}_{a_t, \tau_{t+1:T} \sim p_{\theta}(\tau)} \left[\nabla_{\theta'} \log(\pi_{\theta'}(a_t | s_t)) \left(\prod_{t''=t}^{t'-1} \frac{\pi_{\theta'}(a_{t''} | s_{t''})}{\pi_{\theta}(a_{t''} | s_{t''})} \right) \mathbb{E}_{a_{t'} \sim p_{\theta}(\tau)} \left[\left(\frac{\pi_{\theta'}(a_{t'} | s_{t'})}{\pi_{\theta}(a_{t'} | s_{t'})} \right) \right] \middle| \tau_{1:t-1}, s_{t'} \right] \middle| \tau_{1:t-1}, s_t \\
&= \mathbb{E}_{a_t, \tau_{t+1:T} \sim p_{\theta}(\tau)} \left[\nabla_{\theta'} \log(\pi_{\theta'}(a_t | s_t)) \left(\prod_{t''=t}^{t'-1} \frac{\pi_{\theta'}(a_{t''} | s_{t''})}{\pi_{\theta}(a_{t''} | s_{t''})} \right) * 1 \right] \middle| \tau_{1:t-1}, s_t \\
&= \mathbb{E}_{a_t, \tau_{t+1:T} \sim p_{\theta}(\tau)} \left[\nabla_{\theta'} \log(\pi_{\theta'}(a_t | s_t)) \left(\prod_{t''=t}^{t'-2} \frac{\pi_{\theta'}(a_{t''} | s_{t''})}{\pi_{\theta}(a_{t''} | s_{t''})} \right) \right] \middle| \tau_{1:t-1}, s_t \\
&= \dots \\
&= \mathbb{E}_{a_t, \tau_{t+1:T} \sim p_{\theta}(\tau)} \left[\nabla_{\theta'} \log(\pi_{\theta'}(a_t | s_t)) \frac{\pi_{\theta'}(a_t | s_t)}{\pi_{\theta}(a_t | s_t)} \right] \middle| \tau_{1:t-1}, s_t \\
&= \mathbb{E}_{a_t \sim p_{\theta}(\tau)} \left[\nabla_{\theta'} \log(\pi_{\theta'}(a_t | s_t)) \frac{\pi_{\theta'}(a_t | s_t)}{\pi_{\theta}(a_t | s_t)} \right] \middle| \tau_{1:t-1}, s_t \\
&= \mathbb{E}_{a_t \sim p_{\theta'}(\tau)} [\nabla_{\theta'} \log(\pi_{\theta'}(a_t | s_t)) | \tau_{1:t-1}, s_t] \\
&= 0
\end{aligned}$$

Therefore

$$\mathbb{E}_{\tau_{1:T} \sim p_{\theta}(\tau)} \left[\nabla_{\theta'} \log(\pi_{\theta'}(a_t | s_t)) \left(\prod_{t''=1}^{t'} \frac{\pi_{\theta'}(a_{t''} | s_{t''})}{\pi_{\theta}(a_{t''} | s_{t''})} \right) f(s_t) \right] = 0$$

and we get that

$$\begin{aligned}
\nabla_{\theta'} J(\theta') &= \nabla_{\theta'} \mathbb{E}_{\tau \sim p_{\theta}(\tau)} \left[\frac{p_{\theta'}(\tau)}{p_{\theta}(\tau)} \sum_{t'=1}^T r(s_{t'}, a_{t'}) \right] \\
&= \mathbb{E}_{\tau_1:T \sim p_{\theta}(\tau)} \left[\sum_{t=1}^T \nabla_{\theta'} \log(\pi_{\theta'}(a_t | s_t)) \left(\prod_{t''=1}^t \frac{\pi_{\theta'}(a_{t''} | s_{t''})}{\pi_{\theta}(a_{t''} | s_{t''})} \right) \left(\sum_{t'=t}^T r(s_{t'}, a_{t'}) \left(\prod_{t''=t+1}^{t'} \frac{\pi_{\theta'}(a_{t''} | s_{t''})}{\pi_{\theta}(a_{t''} | s_{t''})} \right) \right) \right] \\
&= \mathbb{E}_{\tau_1:T \sim p_{\theta}(\tau)} \left[\sum_{t=1}^T \nabla_{\theta'} \log(\pi_{\theta'}(a_t | s_t)) \left(\prod_{t''=1}^t \frac{\pi_{\theta'}(a_{t''} | s_{t''})}{\pi_{\theta}(a_{t''} | s_{t''})} \right) \left(\left(\sum_{t'=t}^T r(s_{t'}, a_{t'}) \right) + f(s_t) \right) \left(\prod_{t''=t+1}^{t'} \frac{\pi_{\theta'}(a_{t''} | s_{t''})}{\pi_{\theta}(a_{t''} | s_{t''})} \right) \right]
\end{aligned}$$

10 DPO

In RLHF, we train a reward model on top of the reference model (typically the SFT model) samples. When training the reward model, we give the reference model some prompts and generate a pair of responses per prompt. We then ask humans to compare the pairs of responses and provide labels for which one is better. This gives us a data set $\mathcal{D}$ with samples of the form (x, y_w, y_l) where y_w is the preferred response and y_l is the dis-preferred response. The reward model r_ϕ takes in a (prompt, response) pair and outputs a reward, $r_\phi(\text{prompt}, \text{response})$ for how good the response is for the prompt. A ranking loss (pairwise logistic loss) is used during training on the samples, which tries to make $r_\phi(x, y_w)$ higher than $r_\phi(x, y_l)$ for each sample. This is done through the Bradley Terry model that sets

$$\begin{aligned} p(r_\phi(x, y_w) > r_\phi(x, y_l)) &= \sigma(r_\phi(x, y_w) - r_\phi(x, y_l)) \\ &= \frac{1}{1 + \exp(-(r_\phi(x, y_w) - r_\phi(x, y_l)))} = \frac{\exp(r_\phi(x, y_w))}{1 + \exp(r_\phi(x, y_l))} \end{aligned}$$

giving us a reward model objective of

$$\mathcal{L}_R = -\mathbb{E}_{(x, y_w, y_l) \sim \mathcal{D}} [\log \sigma(r_\phi(x, y_w) - r_\phi(x, y_l))]$$

After training this reward model r_ϕ , we then optimize the SFT model π_θ using RL with the following objective

$$\max_{\pi_\theta} \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_\theta(y|x)} [r_\phi(x, y)] - \beta D_{\text{KL}}[\pi_\theta(y|x) || \pi_{\text{ref}}(y|x)]$$

where $\mathcal{D}$ is the a new dataset of prompts from what was used to train the reward model (this causes some distributional shift). To mitigate further distributional shift and so that the LLM does not take advantage of unrealistic or inaccurate rewards produced by the reward model, we want the LLM to produce responses close to the reference model used to train the reward model, which is why we add a KL divergence penalty. We optimize this objective typically through PPO. However, this is complex, requiring an extra reward model. Moreover, the reward model rewards are not comparable across prompts since the loss only cares about the difference in rewards, so increasing all rewards for a particular prompt by a constant does not change the loss.

Instead of training a reward model and then training the LLM with PPO, we can directly try making the LLM maximize the rewards of the responses without a separate reward model.

As derived in the "Explicit Policy Constraints" section, the closed form solution of

$$\max_{\pi_\theta} \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_\theta(y|x)} [r_\phi(x, y)] - \beta D_{\text{KL}}[\pi_\theta(y|x) || \pi_{\text{ref}}(y|x)]$$

for any particular reward function r_ϕ can be expressed as

$$\pi_\theta^*(y|x) = \frac{1}{Z(x)} \pi_{\text{ref}}(y|x) \exp\left(\frac{r_\phi(x, y)}{\beta}\right)$$

where

$$Z(x) = \int y \pi_{\text{ref}}(y|x) \exp\left(\frac{r_\phi(x, y)}{\beta}\right) dy$$

solving for r_ϕ , we get

$$\begin{aligned} \pi_\theta^*(y|x) &= \frac{1}{Z(x)} \pi_{\text{ref}}(y|x) \exp\left(\frac{r_\phi(x, y)}{\beta}\right) \\ \implies \exp\left(\frac{r_\phi(x, y)}{\beta}\right) &= \frac{Z(x) \pi_\theta^*(y|x)}{\pi_{\text{ref}}(y|x)} \\ \implies r_\phi(x, y) &= \beta \log \frac{\pi_\theta^*(y|x)}{\pi_{\text{ref}}(y|x)} + \beta \log Z(x) \end{aligned}$$

Given any arbitrary policy π^* , this expression represents the reward function that π^* optimizes. We can then directly plug this into our reward model objective to get a loss that can directly optimize our policy (e.g. LLM)

$$\mathcal{L}_{DPO} = -\mathbb{E}_{(x, y_w, y_l) \sim \mathcal{D}} \left[\log \sigma \left(\beta \log \frac{\pi_\theta(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta \log \frac{\pi_\theta(y_l|x)}{\pi_{\text{ref}}(y_l|x)} \right) \right]$$

Maximizing this expression essentially finds us a policy such that the reward function that the policy optimizes maximizes the expected log probability that the reward is higher for the preferred response. The gradient of the expression can be expressed as

$$\begin{aligned}
\nabla_{\theta} \mathcal{L}_{DPO} &= -\mathbb{E}_{(x, y_w, y_l) \sim \mathcal{D}} \left[\nabla_{\theta} \log \sigma \left(\beta \log \frac{\pi_{\theta}(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta \log \frac{\pi_{\theta}(y_l|x)}{\pi_{\text{ref}}(y_l|x)} \right) \right] \\
&= -\mathbb{E}_{(x, y_w, y_l) \sim \mathcal{D}} \left[\frac{1}{\sigma \left(\beta \log \frac{\pi_{\theta}(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta \log \frac{\pi_{\theta}(y_l|x)}{\pi_{\text{ref}}(y_l|x)} \right)} \nabla_{\theta} \sigma \left(\beta \log \frac{\pi_{\theta}(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta \log \frac{\pi_{\theta}(y_l|x)}{\pi_{\text{ref}}(y_l|x)} \right) \right] \\
&= -\mathbb{E}_{(x, y_w, y_l) \sim \mathcal{D}} \left[\frac{1}{\sigma \left(\beta \log \frac{\pi_{\theta}(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta \log \frac{\pi_{\theta}(y_l|x)}{\pi_{\text{ref}}(y_l|x)} \right)} \sigma \left(\beta \log \frac{\pi_{\theta}(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta \log \frac{\pi_{\theta}(y_l|x)}{\pi_{\text{ref}}(y_l|x)} \right) \right. \\
&\quad \left. \left(1 - \sigma \left(\beta \log \frac{\pi_{\theta}(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta \log \frac{\pi_{\theta}(y_l|x)}{\pi_{\text{ref}}(y_l|x)} \right) \right) \nabla_{\theta} \left(\beta \log \frac{\pi_{\theta}(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta \log \frac{\pi_{\theta}(y_l|x)}{\pi_{\text{ref}}(y_l|x)} \right) \right] \\
&= -\mathbb{E}_{(x, y_w, y_l) \sim \mathcal{D}} \left[\left(1 - \sigma \left(\beta \log \frac{\pi_{\theta}(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta \log \frac{\pi_{\theta}(y_l|x)}{\pi_{\text{ref}}(y_l|x)} \right) \right) \beta \nabla_{\theta} (\log \pi_{\theta}(y_w|x) - \log \pi_{\theta}(y_l|x)) \right] \\
&= -\beta \mathbb{E}_{(x, y_w, y_l) \sim \mathcal{D}} \left[\left(\sigma \left(\beta \log \frac{\pi_{\theta}(y_l|x)}{\pi_{\text{ref}}(y_l|x)} - \beta \log \frac{\pi_{\theta}(y_w|x)}{\pi_{\text{ref}}(y_w|x)} \right) \right) \nabla_{\theta} (\log \pi_{\theta}(y_w|x) - \log \pi_{\theta}(y_l|x)) \right] \\
&= -\beta \mathbb{E}_{(x, y_w, y_l) \sim \mathcal{D}} \left[\left(\sigma \left(\beta \log \frac{\pi_{\theta}(y_l|x)}{\pi_{\text{ref}}(y_l|x)} - \beta \log \frac{\pi_{\theta}(y_w|x)}{\pi_{\text{ref}}(y_w|x)} \right) \right) \nabla_{\theta} (\log \pi_{\theta}(y_w|x) - \log \pi_{\theta}(y_l|x)) \right] \\
&= -\beta \mathbb{E}_{(x, y_w, y_l) \sim \mathcal{D}} \left[(\sigma(r(x, y_l) - r(x, y_w))) \nabla_{\theta} (\log \pi_{\theta}(y_w|x) - \log \pi_{\theta}(y_l|x)) \right]
\end{aligned}$$

where we basically just use the chain rule, $1 - \sigma(s) = \sigma(-s)$ and $\frac{\partial}{\partial s} \sigma(s) = \sigma(s)(1 - \sigma(s))$. At each training step, we subtract the gradient above (so the negative in front goes away). Intuitively, we can think of $\nabla_{\theta} (\log \pi_{\theta}(y_w|x) - \log \pi_{\theta}(y_l|x))$ as maximizing the probability of the preferred response and $\sigma(r(x, y_l) - r(x, y_w))$ gives more weight to examples where the implicit reward the policy maximizes is incorrect. Note that

$$\nabla_{\theta} (\log \pi_{\theta}(y_w|x) - \log \pi_{\theta}(y_l|x))$$

causes the gradient to be zero for the first few tokens if they are the same between the preferred and dispreferred responses, meaning DPO tends to learn the difference between the good and bad responses rather than trying to output something specific.

11 Off Policy Policy Gradient Comparison

From the "Advanced Policy Gradients" section, we have that

$$\begin{aligned} J(\theta') - J(\theta) &= \mathbb{E}_{\tau \sim p_{\theta'}(\tau)} \left[\sum_{t=0}^{\infty} \gamma^t (r(s_t, a_t) + \gamma V^{\pi_{\theta}}(s_{t+1}) - V^{\pi_{\theta}}(s_t)) \right] \\ &= \mathbb{E}_{\tau \sim p_{\theta'}(\tau)} \left[\sum_{t=0}^{\infty} \gamma^t A^{\pi_{\theta}}(s_t, a_t) \right] \end{aligned}$$

Removing the discount factor, we get

$$J(\theta') - J(\theta) = \mathbb{E}_{\tau \sim p_{\theta'}(\tau)} \left[\sum_{t=1}^{\infty} A^{\pi_{\theta}}(s_t, a_t) \right]$$

Treating $A^{\pi_{\theta}}(s_t, a_t)$ as $r(s_t, a_t)$ in the derivation of the off policy policy gradient, we get that

$$\begin{aligned} \nabla_{\theta'} J(\theta') &= \nabla_{\theta'} (J(\theta') - J(\theta)) \\ &= \nabla_{\theta'} \mathbb{E}_{\tau \sim p_{\theta'}(\tau)} \left[\sum_{t=1}^{\infty} A^{\pi_{\theta}}(s_t, a_t) \right] \\ &= \mathbb{E}_{\tau_{1:\infty} \sim p_{\theta}(\tau)} \left[\sum_{t=1}^{\infty} \nabla_{\theta'} \log(\pi_{\theta'}(a_t | s_t)) \left(\prod_{t''=1}^t \frac{\pi_{\theta'}(a_{t''} | s_{t''})}{\pi_{\theta}(a_{t''} | s_{t''})} \right) \left(\sum_{t'=t}^{\infty} A^{\pi_{\theta}}(s_{t'}, a_{t'}) \left(\prod_{t''=t+1}^{t'} \frac{\pi_{\theta'}(a_{t''} | s_{t''})}{\pi_{\theta}(a_{t''} | s_{t''})} \right) \right) \right] \\ &= \sum_{t=1}^{\infty} \sum_{t'=t}^{\infty} \mathbb{E}_{\tau_{1:\infty} \sim p_{\theta}(\tau)} \left[\nabla_{\theta'} \log(\pi_{\theta'}(a_t | s_t)) A^{\pi_{\theta}}(s_{t'}, a_{t'}) \left(\prod_{t''=1}^{t'} \frac{\pi_{\theta'}(a_{t''} | s_{t''})}{\pi_{\theta}(a_{t''} | s_{t''})} \right) \right] \end{aligned}$$

Looking at each individual term in the sum, let's look at

Case 1: $t' = t$

$$\begin{aligned} &\mathbb{E}_{\tau_{1:\infty} \sim p_{\theta}(\tau)} \left[\nabla_{\theta'} \log(\pi_{\theta'}(a_t | s_t)) A^{\pi_{\theta}}(s_t, a_t) \left(\prod_{t''=1}^t \frac{\pi_{\theta'}(a_{t''} | s_{t''})}{\pi_{\theta}(a_{t''} | s_{t''})} \right) \right] \\ &= \mathbb{E}_{\tau_{1:\infty} \sim p_{\theta}(\tau)} \left[\nabla_{\theta'} \log(\pi_{\theta'}(a_t | s_t)) (Q^{\pi_{\theta}}(s_t, a_t) - V^{\pi_{\theta}}(s_t)) \left(\prod_{t''=1}^t \frac{\pi_{\theta'}(a_{t''} | s_{t''})}{\pi_{\theta}(a_{t''} | s_{t''})} \right) \right] \\ &= \mathbb{E}_{\tau_{1:\infty} \sim p_{\theta}(\tau)} \left[\nabla_{\theta'} \log(\pi_{\theta'}(a_t | s_t)) Q^{\pi_{\theta}}(s_t, a_t) \left(\prod_{t''=1}^t \frac{\pi_{\theta'}(a_{t''} | s_{t''})}{\pi_{\theta}(a_{t''} | s_{t''})} \right) \right] \\ &= \mathbb{E}_{\tau_{1:\infty} \sim p_{\theta}(\tau)} \left[\nabla_{\theta'} \log(\pi_{\theta'}(a_t | s_t)) \mathbb{E}_{\tau_{t+1:\infty} \sim p_{\theta}(\tau)} \left[\left(\sum_{t'=t}^{\infty} r(s_{t'}, a_{t'}) \right) \middle| s_t, a_t \right] \left(\prod_{t''=1}^t \frac{\pi_{\theta'}(a_{t''} | s_{t''})}{\pi_{\theta}(a_{t''} | s_{t''})} \right) \right] \\ &= \mathbb{E}_{\tau_{1:\infty} \sim p_{\theta}(\tau)} \left[\nabla_{\theta'} \log(\pi_{\theta'}(a_t | s_t)) \mathbb{E}_{\tau_{t+1:\infty} \sim p_{\theta}(\tau)} \left[\left(\sum_{t'=t}^{\infty} r(s_{t'}, a_{t'}) \right) \middle| \tau_{1:t-1}, s_t, a_t \right] \left(\prod_{t''=1}^t \frac{\pi_{\theta'}(a_{t''} | s_{t''})}{\pi_{\theta}(a_{t''} | s_{t''})} \right) \right] \\ &= \mathbb{E}_{\tau_{1:\infty} \sim p_{\theta}(\tau)} \left[\mathbb{E}_{\tau_{t+1:\infty} \sim p_{\theta}(\tau)} \left[\nabla_{\theta'} \log(\pi_{\theta'}(a_t | s_t)) \left(\sum_{t'=t}^{\infty} r(s_{t'}, a_{t'}) \right) \left(\prod_{t''=1}^t \frac{\pi_{\theta'}(a_{t''} | s_{t''})}{\pi_{\theta}(a_{t''} | s_{t''})} \right) \middle| \tau_{1:t-1}, s_t, a_t \right] \right] \\ &= \mathbb{E}_{\tau_{1:\infty} \sim p_{\theta}(\tau)} \left[\nabla_{\theta'} \log(\pi_{\theta'}(a_t | s_t)) \left(\prod_{t''=1}^t \frac{\pi_{\theta'}(a_{t''} | s_{t''})}{\pi_{\theta}(a_{t''} | s_{t''})} \right) \left(\sum_{t'=t}^{\infty} r(s_{t'}, a_{t'}) \right) \right] \\ &= \mathbb{E}_{\tau_{1:\infty} \sim p_{\theta}(\tau)} \left[\nabla_{\theta'} \log(\pi_{\theta'}(a_t | s_t)) \left(\sum_{t'=t}^{\infty} r(s_{t'}, a_{t'}) \right) \left(\prod_{t''=1}^t \frac{\pi_{\theta'}(a_{t''} | s_{t''})}{\pi_{\theta}(a_{t''} | s_{t''})} \right) \right] \end{aligned}$$

Now let's consider

Case 2: $t' > t$:

$$\begin{aligned}
& \mathbb{E}_{\tau_{1:\infty} \sim p_\theta(\tau)} \left[\nabla_{\theta'} \log(\pi_{\theta'}(a_t|s_t)) A^{\pi_\theta}(s_{t'}, a_{t'}) \left(\prod_{t''=1}^{t'} \frac{\pi_{\theta'}(a_{t''}|s_{t''})}{\pi_\theta(a_{t''}|s_{t''})} \right) \right] \\
&= \mathbb{E}_{\tau_{1:\infty} \sim p_\theta(\tau)} \left[\nabla_{\theta'} \log(\pi_{\theta'}(a_t|s_t)) (Q^{\pi_\theta}(s_{t'}, a_{t'}) - V^{\pi_\theta}(s_{t'})) \left(\prod_{t''=1}^{t'} \frac{\pi_{\theta'}(a_{t''}|s_{t''})}{\pi_\theta(a_{t''}|s_{t''})} \right) \right] \\
&= \mathbb{E}_{\tau_{1:\infty} \sim p_\theta(\tau)} \left[\nabla_{\theta'} \log(\pi_{\theta'}(a_t|s_t)) Q^{\pi_\theta}(s_{t'}, a_{t'}) \left(\prod_{t''=1}^{t'} \frac{\pi_{\theta'}(a_{t''}|s_{t''})}{\pi_\theta(a_{t''}|s_{t''})} \right) \right] \\
&\quad - \mathbb{E}_{\tau_{1:\infty} \sim p_\theta(\tau)} \left[\nabla_{\theta'} \log(\pi_{\theta'}(a_t|s_t)) V^{\pi_\theta}(s_{t'}) \left(\prod_{t''=1}^{t'} \frac{\pi_{\theta'}(a_{t''}|s_{t''})}{\pi_\theta(a_{t''}|s_{t''})} \right) \right]
\end{aligned}$$

Analyzing the Q term:

$$\begin{aligned}
& \mathbb{E}_{\tau_{1:\infty} \sim p_\theta(\tau)} \left[\nabla_{\theta'} \log(\pi_{\theta'}(a_t|s_t)) Q^{\pi_\theta}(s_{t'}, a_{t'}) \left(\prod_{t''=1}^{t'} \frac{\pi_{\theta'}(a_{t''}|s_{t''})}{\pi_\theta(a_{t''}|s_{t''})} \right) \right] \\
&= \mathbb{E}_{\tau_{1:t'-1} \sim p_\theta(\tau)} \left[\mathbb{E}_{\tau_{t':\infty} \sim p_\theta(\tau)} \left[\nabla_{\theta'} \log(\pi_{\theta'}(a_t|s_t)) Q^{\pi_\theta}(s_{t'}, a_{t'}) \left(\prod_{t''=1}^{t'} \frac{\pi_{\theta'}(a_{t''}|s_{t''})}{\pi_\theta(a_{t''}|s_{t''})} \right) \middle| \tau_{1:t'-1} \right] \right] \\
&= \mathbb{E}_{\tau_{1:t'-1} \sim p_\theta(\tau)} \left[\nabla_{\theta'} \log(\pi_{\theta'}(a_t|s_t)) \left(\prod_{t''=1}^{t'-1} \frac{\pi_{\theta'}(a_{t''}|s_{t''})}{\pi_\theta(a_{t''}|s_{t''})} \right) \mathbb{E}_{\tau_{t':\infty} \sim p_\theta(\tau)} \left[Q^{\pi_\theta}(s_{t'}, a_{t'}) \frac{\pi_{\theta'}(a_{t'}|s_{t'})}{\pi_\theta(a_{t'}|s_{t'})} \middle| \tau_{1:t'-1} \right] \right]
\end{aligned}$$

Notice how we can separate $\nabla_{\theta'} \log(\pi_{\theta'}(a_t|s_t))$ and $Q^{\pi_\theta}(s_{t'}, a_{t'})$ since $t' > t$. Looking at the inner expectation of the Q term:

$$\begin{aligned}
& \mathbb{E}_{\tau_{t':\infty} \sim p_\theta(\tau)} \left[Q^{\pi_\theta}(s_{t'}, a_{t'}) \frac{\pi_{\theta'}(a_{t'}|s_{t'})}{\pi_\theta(a_{t'}|s_{t'})} \middle| \tau_{1:t'-1} \right] \\
&= \mathbb{E}_{\tau_{t':\infty} \sim p_\theta(\tau)} \left[\frac{\pi_{\theta'}(a_{t'}|s_{t'})}{\pi_\theta(a_{t'}|s_{t'})} \mathbb{E}_{\tau_{t'+1:\infty} \sim p_\theta(\tau)} \left[\sum_{t'''=t'}^{\infty} r(s_{t'''}, a_{t'''}) \middle| s_{t'}, a_{t'} \right] \middle| \tau_{1:t'-1} \right] \\
&= \mathbb{E}_{\tau_{t':\infty} \sim p_\theta(\tau)} \left[\mathbb{E}_{\tau_{t'+1:\infty} \sim p_\theta(\tau)} \left[\frac{\pi_{\theta'}(a_{t'}|s_{t'})}{\pi_\theta(a_{t'}|s_{t'})} \sum_{t'''=t'}^{\infty} r(s_{t'''}, a_{t'''}) \middle| \tau_{1:t'-1}, s_{t'}, a_{t'} \right] \middle| \tau_{1:t'-1} \right] \\
&= \mathbb{E}_{\tau_{t':\infty} \sim p_\theta(\tau)} \left[\frac{\pi_{\theta'}(a_{t'}|s_{t'})}{\pi_\theta(a_{t'}|s_{t'})} \sum_{t'''=t'}^{\infty} r(s_{t'''}, a_{t'''}) \middle| \tau_{1:t'-1} \right]
\end{aligned}$$

Plugging this into the Q term, we get:

$$\begin{aligned}
& \mathbb{E}_{\tau_{1:\infty} \sim p_\theta(\tau)} \left[\nabla_{\theta'} \log(\pi_{\theta'}(a_t|s_t)) Q^{\pi_\theta}(s_{t'}, a_{t'}) \left(\prod_{t''=1}^{t'} \frac{\pi_{\theta'}(a_{t''}|s_{t''})}{\pi_\theta(a_{t''}|s_{t''})} \right) \right] \\
&= \mathbb{E}_{\tau_{1:t'-1} \sim p_\theta(\tau)} \left[\nabla_{\theta'} \log(\pi_{\theta'}(a_t|s_t)) \left(\prod_{t''=1}^{t'-1} \frac{\pi_{\theta'}(a_{t''}|s_{t''})}{\pi_\theta(a_{t''}|s_{t''})} \right) \mathbb{E}_{\tau_{t':\infty} \sim p_\theta(\tau)} \left[Q^{\pi_\theta}(s_{t'}, a_{t'}) \frac{\pi_{\theta'}(a_{t'}|s_{t'})}{\pi_\theta(a_{t'}|s_{t'})} \middle| \tau_{1:t'-1} \right] \right] \\
&= \mathbb{E}_{\tau_{1:t'-1} \sim p_\theta(\tau)} \left[\nabla_{\theta'} \log(\pi_{\theta'}(a_t|s_t)) \left(\prod_{t''=1}^{t'-1} \frac{\pi_{\theta'}(a_{t''}|s_{t''})}{\pi_\theta(a_{t''}|s_{t''})} \right) \mathbb{E}_{\tau_{t':\infty} \sim p_\theta(\tau)} \left[\frac{\pi_{\theta'}(a_{t'}|s_{t'})}{\pi_\theta(a_{t'}|s_{t'})} \sum_{t'''=t'}^{\infty} r(s_{t'''}, a_{t'''}) \middle| \tau_{1:t'-1} \right] \right] \\
&= \mathbb{E}_{\tau_{1:t'-1} \sim p_\theta(\tau)} \left[\mathbb{E}_{\tau_{t':\infty} \sim p_\theta(\tau)} \left[\nabla_{\theta'} \log(\pi_{\theta'}(a_t|s_t)) \left(\prod_{t''=1}^{t'} \frac{\pi_{\theta'}(a_{t''}|s_{t''})}{\pi_\theta(a_{t''}|s_{t''})} \right) \sum_{t'''=t'}^{\infty} r(s_{t'''}, a_{t'''}) \middle| \tau_{1:t'-1} \right] \right] \\
&= \mathbb{E}_{\tau_{1:\infty} \sim p_\theta(\tau)} \left[\nabla_{\theta'} \log(\pi_{\theta'}(a_t|s_t)) \left(\prod_{t''=1}^{t'} \frac{\pi_{\theta'}(a_{t''}|s_{t''})}{\pi_\theta(a_{t''}|s_{t''})} \right) \sum_{t'''=t'}^{\infty} r(s_{t'''}, a_{t'''}) \right]
\end{aligned}$$

Analyzing the V term:

$$\begin{aligned}
& \mathbb{E}_{\tau_{1:\infty} \sim p_\theta(\tau)} \left[\nabla_{\theta'} \log(\pi_{\theta'}(a_t | s_t)) V^{\pi_\theta}(s_{t'}) \left(\prod_{t''=1}^{t'} \frac{\pi_{\theta'}(a_{t''} | s_{t''})}{\pi_\theta(a_{t''} | s_{t''})} \right) \right] \\
= & \mathbb{E}_{\tau_{1:t'-1}, s_{t'} \sim p_\theta(\tau)} \left[\mathbb{E}_{a_{t'}, \tau_{t'+1:\infty} \sim p_\theta(\tau)} \left[\nabla_{\theta'} \log(\pi_{\theta'}(a_t | s_t)) V^{\pi_\theta}(s_{t'}) \left(\prod_{t''=1}^{t'} \frac{\pi_{\theta'}(a_{t''} | s_{t''})}{\pi_\theta(a_{t''} | s_{t''})} \right) \middle| \tau_{1:t'-1}, s_{t'} \right] \right] \\
= & \mathbb{E}_{\tau_{1:t'-1}, s_{t'} \sim p_\theta(\tau)} \left[\nabla_{\theta'} \log(\pi_{\theta'}(a_t | s_t)) \left(\prod_{t''=1}^{t'-1} \frac{\pi_{\theta'}(a_{t''} | s_{t''})}{\pi_\theta(a_{t''} | s_{t''})} \right) V^{\pi_\theta}(s_{t'}) \mathbb{E}_{a_{t'}, \tau_{t'+1:\infty} \sim p_\theta(\tau)} \left[\frac{\pi_{\theta'}(a_{t'} | s_{t'})}{\pi_\theta(a_{t'} | s_{t'})} \middle| \tau_{1:t'-1}, s_{t'} \right] \right] \\
= & \mathbb{E}_{\tau_{1:t'-1}, s_{t'} \sim p_\theta(\tau)} \left[\nabla_{\theta'} \log(\pi_{\theta'}(a_t | s_t)) \left(\prod_{t''=1}^{t'-1} \frac{\pi_{\theta'}(a_{t''} | s_{t''})}{\pi_\theta(a_{t''} | s_{t''})} \right) V^{\pi_\theta}(s_{t'}) \mathbb{E}_{a_{t'} \sim \pi_{\theta'}(a_{t'} | s_{t'})} \left[\frac{\pi_{\theta'}(a_{t'} | s_{t'})}{\pi_\theta(a_{t'} | s_{t'})} \middle| \tau_{1:t'-1}, s_{t'} \right] \right] \\
= & \mathbb{E}_{\tau_{1:t'-1}, s_{t'} \sim p_\theta(\tau)} \left[\nabla_{\theta'} \log(\pi_{\theta'}(a_t | s_t)) \left(\prod_{t''=1}^{t'-1} \frac{\pi_{\theta'}(a_{t''} | s_{t''})}{\pi_\theta(a_{t''} | s_{t''})} \right) V^{\pi_\theta}(s_{t'}) * 1 \right] \\
= & \mathbb{E}_{\tau_{1:t'-1}, s_{t'} \sim p_\theta(\tau)} \left[\nabla_{\theta'} \log(\pi_{\theta'}(a_t | s_t)) \left(\prod_{t''=1}^{t'-1} \frac{\pi_{\theta'}(a_{t''} | s_{t''})}{\pi_\theta(a_{t''} | s_{t''})} \right) V^{\pi_\theta}(s_{t'}) \right] \\
= & \mathbb{E}_{\tau_{1:t'-1}, s_{t'} \sim p_\theta(\tau)} \left[\nabla_{\theta'} \log(\pi_{\theta'}(a_t | s_t)) \left(\prod_{t''=1}^{t'-1} \frac{\pi_{\theta'}(a_{t''} | s_{t''})}{\pi_\theta(a_{t''} | s_{t''})} \right) \mathbb{E}_{a_{t'}, \tau_{t'+1:\infty} \sim p_\theta(\tau)} \left[\sum_{t'''=t'}^{\infty} r(s_{t'''}, a_{t'''}) \middle| s_{t'} \right] \right] \\
= & \mathbb{E}_{\tau_{1:t'-1}, s_{t'} \sim p_\theta(\tau)} \left[\nabla_{\theta'} \log(\pi_{\theta'}(a_t | s_t)) \left(\prod_{t''=1}^{t'-1} \frac{\pi_{\theta'}(a_{t''} | s_{t''})}{\pi_\theta(a_{t''} | s_{t''})} \right) \mathbb{E}_{a_{t'}, \tau_{t'+1:\infty} \sim p_\theta(\tau)} \left[\sum_{t'''=t'}^{\infty} r(s_{t'''}, a_{t'''}) \middle| \tau_{1:t'-1}, s_{t'} \right] \right] \\
= & \mathbb{E}_{\tau_{1:t'-1}, s_{t'} \sim p_\theta(\tau)} \left[\mathbb{E}_{a_{t'}, \tau_{t'+1:\infty} \sim p_\theta(\tau)} \left[\nabla_{\theta'} \log(\pi_{\theta'}(a_t | s_t)) \left(\prod_{t''=1}^{t'-1} \frac{\pi_{\theta'}(a_{t''} | s_{t''})}{\pi_\theta(a_{t''} | s_{t''})} \right) \sum_{t'''=t'}^{\infty} r(s_{t'''}, a_{t'''}) \middle| \tau_{1:t'-1}, s_{t'} \right] \right] \\
= & \mathbb{E}_{\tau_{1:\infty} \sim p_\theta(\tau)} \left[\nabla_{\theta'} \log(\pi_{\theta'}(a_t | s_t)) \left(\prod_{t''=1}^{t'-1} \frac{\pi_{\theta'}(a_{t''} | s_{t''})}{\pi_\theta(a_{t''} | s_{t''})} \right) \sum_{t'''=t'}^{\infty} r(s_{t'''}, a_{t'''}) \right]
\end{aligned}$$

Plugging in the Q and V terms into the individual term in the sum where $t' > t$:

$$\begin{aligned}
& \mathbb{E}_{\tau_{1:\infty} \sim p_\theta(\tau)} \left[\nabla_{\theta'} \log(\pi_{\theta'}(a_t | s_t)) A^{\pi_\theta}(s_{t'}, a_{t'}) \left(\prod_{t''=1}^{t'} \frac{\pi_{\theta'}(a_{t''} | s_{t''})}{\pi_\theta(a_{t''} | s_{t''})} \right) \right] \\
= & \mathbb{E}_{\tau_{1:\infty} \sim p_\theta(\tau)} \left[\nabla_{\theta'} \log(\pi_{\theta'}(a_t | s_t)) Q^{\pi_\theta}(s_{t'}, a_{t'}) \left(\prod_{t''=1}^{t'} \frac{\pi_{\theta'}(a_{t''} | s_{t''})}{\pi_\theta(a_{t''} | s_{t''})} \right) \right] \\
& - \mathbb{E}_{\tau_{1:\infty} \sim p_\theta(\tau)} \left[\nabla_{\theta'} \log(\pi_{\theta'}(a_t | s_t)) V^{\pi_\theta}(s_{t'}) \left(\prod_{t''=1}^{t'} \frac{\pi_{\theta'}(a_{t''} | s_{t''})}{\pi_\theta(a_{t''} | s_{t''})} \right) \right] \\
= & \mathbb{E}_{\tau_{1:\infty} \sim p_\theta(\tau)} \left[\nabla_{\theta'} \log(\pi_{\theta'}(a_t | s_t)) \left(\prod_{t''=1}^{t'} \frac{\pi_{\theta'}(a_{t''} | s_{t''})}{\pi_\theta(a_{t''} | s_{t''})} \right) \sum_{t'''=t'}^{\infty} r(s_{t'''}, a_{t'''}) \right] \\
& - \mathbb{E}_{\tau_{1:\infty} \sim p_\theta(\tau)} \left[\nabla_{\theta'} \log(\pi_{\theta'}(a_t | s_t)) \left(\prod_{t''=1}^{t'-1} \frac{\pi_{\theta'}(a_{t''} | s_{t''})}{\pi_\theta(a_{t''} | s_{t''})} \right) \sum_{t'''=t'}^{\infty} r(s_{t'''}, a_{t'''}) \right] \\
= & \mathbb{E}_{\tau_{1:\infty} \sim p_\theta(\tau)} \left[\nabla_{\theta'} \log(\pi_{\theta'}(a_t | s_t)) \left(\left(\prod_{t''=1}^{t'} \frac{\pi_{\theta'}(a_{t''} | s_{t''})}{\pi_\theta(a_{t''} | s_{t''})} \right) - \left(\prod_{t''=1}^{t'-1} \frac{\pi_{\theta'}(a_{t''} | s_{t''})}{\pi_\theta(a_{t''} | s_{t''})} \right) \right) \sum_{t'''=t'}^{\infty} r(s_{t'''}, a_{t'''}) \right] \\
= & \mathbb{E}_{\tau_{1:\infty} \sim p_\theta(\tau)} \left[\nabla_{\theta'} \log(\pi_{\theta'}(a_t | s_t)) \left(\sum_{t'''=t'}^{\infty} r(s_{t'''}, a_{t'''}) \left(\left(\prod_{t''=1}^{t'} \frac{\pi_{\theta'}(a_{t''} | s_{t''})}{\pi_\theta(a_{t''} | s_{t''})} \right) - \left(\prod_{t''=1}^{t'-1} \frac{\pi_{\theta'}(a_{t''} | s_{t''})}{\pi_\theta(a_{t''} | s_{t''})} \right) \right) \right) \right]
\end{aligned}$$

Plugging the expressions for the two cases:

$$\begin{aligned}
& \nabla_{\theta'} J(\theta') \\
&= \sum_{t=1}^{\infty} \sum_{t'=t}^{\infty} \mathbb{E}_{\tau_{1:\infty} \sim p_{\theta}(\tau)} \left[\nabla_{\theta'} \log(\pi_{\theta'}(a_t | s_t)) A^{\pi_{\theta}}(s_{t'}, a_{t'}) \left(\prod_{t''=1}^{t'} \frac{\pi_{\theta'}(a_{t''} | s_{t''})}{\pi_{\theta}(a_{t''} | s_{t''})} \right) \right] \\
&= \sum_{t=1}^{\infty} \mathbb{E}_{\tau_{1:\infty} \sim p_{\theta}(\tau)} \left[\nabla_{\theta'} \log(\pi_{\theta'}(a_t | s_t)) \left(\sum_{t'''=t}^{\infty} r(s_{t'''}, a_{t'''}) \left(\prod_{t''=1}^t \frac{\pi_{\theta'}(a_{t''} | s_{t''})}{\pi_{\theta}(a_{t''} | s_{t''})} \right) \right) \right] \\
&\quad + \sum_{t=1}^{\infty} \sum_{t'=t+1}^{\infty} \mathbb{E}_{\tau_{1:\infty} \sim p_{\theta}(\tau)} \left[\nabla_{\theta'} \log(\pi_{\theta'}(a_t | s_t)) \left(\sum_{t'''=t'}^{\infty} r(s_{t'''}, a_{t'''}) \left(\left(\prod_{t''=1}^{t'} \frac{\pi_{\theta'}(a_{t''} | s_{t''})}{\pi_{\theta}(a_{t''} | s_{t''})} \right) - \left(\prod_{t''=1}^{t'-1} \frac{\pi_{\theta'}(a_{t''} | s_{t''})}{\pi_{\theta}(a_{t''} | s_{t''})} \right) \right) \right) \right] \\
&= \sum_{t=1}^{\infty} \sum_{t'''=t}^{\infty} \mathbb{E}_{\tau_{1:\infty} \sim p_{\theta}(\tau)} \left[\nabla_{\theta'} \log(\pi_{\theta'}(a_t | s_t)) \left(r(s_{t'''}, a_{t'''}) \left(\prod_{t''=1}^t \frac{\pi_{\theta'}(a_{t''} | s_{t''})}{\pi_{\theta}(a_{t''} | s_{t''})} \right) \right) \right] \\
&\quad + \sum_{t=1}^{\infty} \sum_{t'=t+1}^{\infty} \sum_{t'''=t'}^{\infty} \mathbb{E}_{\tau_{1:\infty} \sim p_{\theta}(\tau)} \left[\nabla_{\theta'} \log(\pi_{\theta'}(a_t | s_t)) \left(r(s_{t'''}, a_{t'''}) \left(\left(\prod_{t''=1}^{t'} \frac{\pi_{\theta'}(a_{t''} | s_{t''})}{\pi_{\theta}(a_{t''} | s_{t''})} \right) - \left(\prod_{t''=1}^{t'-1} \frac{\pi_{\theta'}(a_{t''} | s_{t''})}{\pi_{\theta}(a_{t''} | s_{t''})} \right) \right) \right) \right] \\
&= \sum_{t=1}^{\infty} \sum_{t'''=t}^{\infty} \mathbb{E}_{\tau_{1:\infty} \sim p_{\theta}(\tau)} \left[\nabla_{\theta'} \log(\pi_{\theta'}(a_t | s_t)) \left(r(s_{t'''}, a_{t'''}) \left(\prod_{t''=1}^t \frac{\pi_{\theta'}(a_{t''} | s_{t''})}{\pi_{\theta}(a_{t''} | s_{t''})} \right) \right) \right] \\
&\quad + \sum_{t=1}^{\infty} \sum_{t'''=t+1}^{\infty} \sum_{t'=t+1}^{t'''} \mathbb{E}_{\tau_{1:\infty} \sim p_{\theta}(\tau)} \left[\nabla_{\theta'} \log(\pi_{\theta'}(a_t | s_t)) \left(r(s_{t'''}, a_{t'''}) \left(\left(\prod_{t''=1}^{t'} \frac{\pi_{\theta'}(a_{t''} | s_{t''})}{\pi_{\theta}(a_{t''} | s_{t''})} \right) - \left(\prod_{t''=1}^{t'-1} \frac{\pi_{\theta'}(a_{t''} | s_{t''})}{\pi_{\theta}(a_{t''} | s_{t''})} \right) \right) \right) \right] \\
&= \sum_{t=1}^{\infty} \sum_{t'''=t}^{\infty} \mathbb{E}_{\tau_{1:\infty} \sim p_{\theta}(\tau)} \left[\nabla_{\theta'} \log(\pi_{\theta'}(a_t | s_t)) \left(r(s_{t'''}, a_{t'''}) \left(\prod_{t''=1}^t \frac{\pi_{\theta'}(a_{t''} | s_{t''})}{\pi_{\theta}(a_{t''} | s_{t''})} \right) \right) \right] \\
&\quad + \sum_{t=1}^{\infty} \sum_{t'''=t+1}^{\infty} \mathbb{E}_{\tau_{1:\infty} \sim p_{\theta}(\tau)} \left[\nabla_{\theta'} \log(\pi_{\theta'}(a_t | s_t)) \left(r(s_{t'''}, a_{t'''}) \sum_{t'=t+1}^{t'''} \left(\left(\prod_{t''=1}^{t'} \frac{\pi_{\theta'}(a_{t''} | s_{t''})}{\pi_{\theta}(a_{t''} | s_{t''})} \right) - \left(\prod_{t''=1}^{t'-1} \frac{\pi_{\theta'}(a_{t''} | s_{t''})}{\pi_{\theta}(a_{t''} | s_{t''})} \right) \right) \right) \right] \\
&= \sum_{t=1}^{\infty} \sum_{t'''=t}^{\infty} \mathbb{E}_{\tau_{1:\infty} \sim p_{\theta}(\tau)} \left[\nabla_{\theta'} \log(\pi_{\theta'}(a_t | s_t)) \left(r(s_{t'''}, a_{t'''}) \right. \right. \\
&\quad \left. \left. * \left(\prod_{t''=1}^t \frac{\pi_{\theta'}(a_{t''} | s_{t''})}{\pi_{\theta}(a_{t''} | s_{t''})} + \sum_{t'=t+1}^{t'''} \left(\left(\prod_{t''=1}^{t'} \frac{\pi_{\theta'}(a_{t''} | s_{t''})}{\pi_{\theta}(a_{t''} | s_{t''})} \right) - \left(\prod_{t''=1}^{t'-1} \frac{\pi_{\theta'}(a_{t''} | s_{t''})}{\pi_{\theta}(a_{t''} | s_{t''})} \right) \right) \right) \right) \right] \\
&= \sum_{t=1}^{\infty} \sum_{t'''=t}^{\infty} \mathbb{E}_{\tau_{1:\infty} \sim p_{\theta}(\tau)} \left[\nabla_{\theta'} \log(\pi_{\theta'}(a_t | s_t)) \left(r(s_{t'''}, a_{t'''}) \left(\prod_{t''=1}^{t'} \frac{\pi_{\theta'}(a_{t''} | s_{t''})}{\pi_{\theta}(a_{t''} | s_{t''})} \right) \right) \right] \\
&= \sum_{t=1}^{\infty} \sum_{t'=t}^{\infty} \mathbb{E}_{\tau_{1:\infty} \sim p_{\theta}(\tau)} \left[\nabla_{\theta'} \log(\pi_{\theta'}(a_t | s_t)) \left(r(s_{t'}, a_{t'}) \left(\prod_{t''=1}^{t'} \frac{\pi_{\theta'}(a_{t''} | s_{t''})}{\pi_{\theta}(a_{t''} | s_{t''})} \right) \right) \right] \\
&= \mathbb{E}_{\tau_{1:\infty} \sim p_{\theta}(\tau)} \left[\sum_{t=1}^{\infty} \nabla_{\theta'} \log(\pi_{\theta'}(a_t | s_t)) \left(\sum_{t'=t}^{\infty} r(s_{t'}, a_{t'}) \left(\prod_{t''=1}^{t'} \frac{\pi_{\theta'}(a_{t''} | s_{t''})}{\pi_{\theta}(a_{t''} | s_{t''})} \right) \right) \right]
\end{aligned}$$

which is the original off policy policy gradient as expected. Note we made use switching summation order ($\sum_{x=c}^{\infty} \sum_{y=x}^{\infty} f(x, y) = \sum_{y=c}^{\infty} \sum_{x=c}^y f(x, y)$) and telescoping sums (can visualize before summation switch by fixing t and manually take the sum incrementing t' one at a time).

12 Q, Value, and Policy Iteration Methods

Defining V and Q recursively:

$$\begin{aligned} V^\pi(s) &= \mathbb{E}_{a \sim \pi(a|s), s' \sim p(s'|s,a)} [r(s, a) + V^\pi(s')|s] \\ Q^\pi(s, a) &= \mathbb{E}_{s' \sim p(s'|s,a), a' \sim \pi(a'|s)} [r(s, a) + Q^\pi(s', a')|s, a] = r(s, a) + \mathbb{E}_{s' \sim p(s'|s,a), a' \sim \pi(a'|s)} [Q^\pi(s', a')] \end{aligned}$$

If the policy is deterministic (used in policy iteration below), we get that

$$\begin{aligned} V^\pi(s) &= \mathbb{E}_{s' \sim p(s'|s, \pi(s))} [r(s, \pi(s)) + V^\pi(s')|s] = r(s, \pi(s)) + \mathbb{E}_{s' \sim p(s'|s, \pi(s))} [V^\pi(s')] \\ Q^\pi(s, a) &= r(s, a) + \mathbb{E}_{s' \sim p(s'|s,a)} [Q^\pi(s', \pi(s'))|s, a] = r(s, a) + \mathbb{E}_{s' \sim p(s'|s,a)} [Q^\pi(s', \pi(s'))] \end{aligned}$$

Defining V and Q as a function of each other:

$$\begin{aligned} V^\pi(s) &= \mathbb{E}_{a \sim \pi(a|s)} [Q^\pi(s, a)|s] \\ Q^\pi(s, a) &= \mathbb{E}_{s' \sim p(s'|s,a)} [r(s, a) + V^\pi(s')|s, a] = r(s, a) + \mathbb{E}_{s' \sim p(s'|s,a)} [V^\pi(s')] \end{aligned}$$

Note that all iteration methods below assume a finite set of actions we can enumerate (needed whenever we take an argmax over a).

12.1 Policy Iteration

12.1.1 Tabular Value Function Variant (Assumes We Know Transition Dynamics)

1.Repeat until policy converges:

1a.For every state s in parallel:

$$1ai.\pi(a|s) = (a == a^* ? 1 : 0), \text{ where } a^* = \underset{a}{\operatorname{argmax}} Q^\pi(s, a) = \underset{a}{\operatorname{argmax}} r(s, a) + \mathbb{E}_{s' \sim p(s'|s,a)} [V^\pi(s')]$$

1b.Repeat until V converges for current policy:

1bi.Repeat for each state s :

$$1bi(1).y_s = r(s, \pi(a|s)) + \mathbb{E}_{s' \sim p(s'|s, \pi(a|s))} [V^\pi(s')]$$

1bii.Repeat for each state s :

$$1bii(1).V^\pi(s) = y_s$$

12.1.2 Tabular Q Function Variant (Assumes We Know Transition Dynamics)

1.Repeat until policy converges:

1a.For every state s in parallel:

$$1ai.\pi(a|s) = (a == a^* ? 1 : 0), \text{ where } a^* = \underset{a}{\operatorname{argmax}} Q^\pi(s, a)$$

1b.Repeat until Q converges for current policy:

1bi.Repeat for each state action pair s, a :

$$1bi(1).y_{s,a} = y_{s,a} = r(s, a) + \mathbb{E}_{s' \sim p(s'|s,a)} [Q^\pi(s', \pi(s'))]$$

1bii.Repeat for each state action pair s, a :

$$1bii(1).Q^\pi(s, a) = y_{s,a}$$

12.1.3 Fitted Value Function Variant Assuming We Know Transition Dynamics

1.Repeat until policy converges or for some steps:

1a.Collect random samples (s_i, a_i, s'_i, r_i) using any policy (possibly current policy with some exploration)

1b.Repeat until policy converges or for some steps:

1bi.For every sampled state s_i in parallel :

$$1bi(1).\pi(a|s_i) = (a == a^* ? 1 : 0), \text{ where } a^* = \underset{a}{\operatorname{argmax}} Q^\pi(s_i, a) = r(s_i, a) + \mathbb{E}_{s' \sim p(s'|s_i, a)} [V^\pi(s')]$$

1bii.Repeat until V_ϕ converges (may not converge) or for some steps:

1bii(1).For all sampled states s_i :

$$1bii(1a).y_i = r(s_i, \pi(s_i)) + \mathbb{E}_{s' \sim p(s'|s_i, \pi(s_i))} [V^\pi(s')]$$

$$1bii(2).\phi = \underset{\phi}{\operatorname{argmin}} \frac{1}{2} \sum_i \|V_\phi(s_i) - y_i\|^2 \text{ (or run k steps of normal or stochastic gradient descent)}$$

12.1.4 Fitted Q Function Variant Assuming We Know Transition Dynamics

1.Repeat until policy converges or for some steps:

1a.Collect random samples (s_i, a_i, s'_i, r_i) using any policy (possibly current policy with some exploration)

1b.Repeat until policy converges or for some steps:

1bi.For every sampled state s_i :

$$1bi(1).\pi(a|s_i) = (a == a^* ? 1 : 0), \text{ where } a^* = \underset{a}{\operatorname{argmax}} Q^\pi(s_i, a)$$

1bii.Repeat until Q_ϕ converges (may not converge) or for some steps:

1bii(1).For all possible pairs of sampled states s_i and actions a in parallel :

$$1bii(1a).y_{i,a} = r(s_i, a) + \mathbb{E}_{s' \sim p(s'|s_i, a)} [Q^\pi(s', \pi(s'))]$$

$$1bii(2).\phi = \underset{\phi}{\operatorname{argmin}} \frac{1}{2} \sum_i \sum_a \|Q_\phi(s_i, a) - y_{i,a}\|^2 \text{ (or run k steps of normal or stochastic gradient descent)}$$

12.1.5 Fitted Value Function Variant Without Transition Dynamics

This is generally not possible, Without the knowledge of the transition dynamics in step 1bi(1) from 8.1.3, we now need to actually try all the different actions from a given state in the environment instead of just computing value for all the actions offline. This is only possible by teleporting back to the same state and trying many actions to estimate $\mathbb{E}_{s' \sim p(s'|s, a_i)} [V^\pi(s')]$. However, this is not possible assuming we can only run policies in the environment.

12.1.6 Fitted Q function Variant Without Transition Dynamics

1.Repeat until policy converges or for some steps:

1a.Collect random samples (s_i, a_i, s'_i, r_i) using any policy (possibly current policy with some exploration)

1b.Repeat until policy converges or for some steps:

1bi.For every sampled state s_i :

$$1bi(1).\pi(a|s_i) = (a == a^* ? 1 : 0), \text{ where } a^* = \underset{a}{\operatorname{argmax}} Q^\pi(s_i, a)$$

1bii.Repeat until Q_ϕ converges (may not converge) or for some steps:

1bii(1).For all samples :

$$1bii(1a).y_i = r_i + Q^\pi(s'_i, \pi(s'_i))$$

$$1bii(2).\phi = \underset{\phi}{\operatorname{argmin}} \frac{1}{2} \sum_i \|Q_\phi(s_i, a_i) - y_i\|^2 \text{ (or run k steps of normal or stochastic gradient descent)}$$

12.1.7 Notes

1. Step 1bi and 1bii in the tabular variants can instead be replaced by solving a system of equations.

2. The fitted variant is for when the state space is too large or continuous. It's inner loop is the exact same as the tabular version except we gather samples first and fit a model instead of updating table values.
3. Setting the label y in the fitted variants is a contraction in the L_∞ space while taking the argmin with respect to ϕ to update the model is a contraction in the L_2 space but the two together is not a contraction of any sort; thus, the loop that repeated sets the labels and takes the argmin with respect to ϕ may never converge.
4. We can technically think of the tabular variant as a version of the fitted variant where the model is just an input embedding vector of length $|S|$ and stores $V(s)$ for the value function variant and length $|S||A|$ and stores $Q(s, a)$ for the Q function variant.
5. We actually discard everything about the sample except for s_i when the transition dynamics $p(s'|s, a)$ are known in the fitted variants.

12.2 Value Iteration

12.2.1 Tabular Variant (Assumes We Know Transition Dynamics)

- 1.Repeat until V converges:
 - 1a.For every state s:
 - 1ai. $y_s = \max_a r(s, a) + \mathbb{E}_{s' \sim p(s'|s, a_i)} [V^\pi(s')]$
 - 1b.For every state s:
 - 1bi. $V^\pi(s) = y_s$

Or alternatively

- 1.Repeat until V converges:
 - 1a.For every state action pair s,a:
 - 1ai. $y_{s,a} = \max_a r(s, a) + \mathbb{E}_{s' \sim p(s'|s, a_i)} [V^\pi(s')]$
 - 1b.For every state action pair s,a:
 - 1bi. $V^\pi(s) = y_s$

12.2.2 Fitted Variant With Transition Dynamics

- 1.Repeat until V_ϕ converges or for some steps:
 - 1a.Collect random samples (s_i, a_i, s'_i, r_i) using any policy (possibly current policy with some exploration)
- 2.Repeat until V_ϕ converges or for some steps:
 - 2a.For every sampled state s_i :
 - 2ai. $y_{s_i} = \max_a r(s_i, a) + \mathbb{E}_{s' \sim p(s'|s_i, a)} [V^\pi(s')]$
 - 2b. $\phi = \operatorname{argmin}_{\phi} \frac{1}{2} \sum_i \|V_\phi(s_i) - y_i\|^2$ (or run k steps of normal or stochastic gradient descent)

12.2.3 Fitted Variant Without Transition Dynamics

This is generally not possible, Without the knowledge of the transition dynamics in step 2ai, we now need to actually try all the different actions from a given state in the environment instead of just computing value for all the actions offline. This is only possible by teleporting back to the same state and trying many actions to estimate $\mathbb{E}_{s' \sim p(s'|s, a_i)} [V^\pi(s')]$. However, this is not possible assuming we can only run policies in the environment.

12.2.4 Notes

1. The tabular variant of value iteration is the same as the value function variant of policy iteration but we do not repeatedly reset the target (only run loop 1b once from policy iteration). Similar observation holds for the fitted

variant knowing transition dynamics.

2. The fitted variant is for when the state space is too large or continuous. It's inner loop is the exact same as the tabular version except we gather samples first and fit a model instead of updating table values.
3. Setting the label y in the fitted variants is a contraction in the L_∞ space while taking the argmin with respect to ϕ to update the model is a contraction in the L_2 space but the two together is not a contraction of any sort; thus, the loop that repeated sets the labels and takes the argmin with respect to ϕ may never converge.
4. We can technically think of the tabular variant as a version of the fitted variant where the model is just an input embedding vector of length $|S|$ and stores $V(s)$.
5. We actually discard everything about the sample except for s_i when the transition dynamics $p(s'|s, a)$ are known in the fitted variant.

12.3 Q Iteration

12.3.1 Tabular Variant (Assumes We Know Transition Dynamics)

1. Repeat until Q converges:
 - 1a. For every state action pair s,a:

$$1ai.y_{s,a} = r(s, a) + \mathbb{E}_{s' \sim p(s'|s, a_i)} \left[\max_{a'} Q^\pi(s', a') \right]$$
 - 1b. For every state action pair s,a:

$$1bi.Q^\pi(s, a) = y_{s,a}$$

12.3.2 Fitted Variant With Transition Dynamics

1. Repeat until Q_ϕ converges or for some steps:
 - 1a. Collect random samples (s_i, a_i, s'_i, r_i) using any policy (possibly current policy with some exploration)
 - 1b. Repeat until Q_ϕ converges or for some steps:
 - 1bi. For all possible pairs of sampled states s_i and actions a :

$$1bi(1).y_{s_i, a} = r(s_i, a) + \mathbb{E}_{s' \sim p(s'|s_i, a)} \left[\max_{a'} Q^\pi(s', a') \right]$$
 - 1bii. $\phi = \operatorname{argmin}_{\phi} \frac{1}{2} \sum_i \|Q_\phi(s_i, a) - y_{i,a}\|^2$ (or run k steps of normal or stochastic gradient descent)

12.3.3 Fitted Variant Without Transition Dynamics

1. Repeat until Q_ϕ converges or for some steps:
 - 1a. Collect random samples (s_i, a_i, s'_i, r_i) using any policy (possibly current policy with some exploration)
 - 1b. Repeat until Q_ϕ converges or for some steps:
 - 1bi. For all samples :

$$1bi(1).y_i = r_i + \max_{a'} Q^\pi(s'_i, a')$$
 - 1bii. $\phi = \operatorname{argmin}_{\phi} \frac{1}{2} \sum_i \|Q_\phi(s_i, a_i) - y_i\|^2$ (or run k steps of normal or stochastic gradient descent)

12.4 Notes

1. The tabular variant of value iteration is the same as the value function variant of policy iteration but we do not repeatedly reset the target (only run loop 1b once from policy iteration). Similar observation holds for the fitted variant knowing transition dynamics.

2. The fitted variant is for when the state space is too large or continuous. It's inner loop is the exact same as the tabular version except we gather samples first and fit a model instead of updating table values.
3. Setting the label y in the fitted variants is a contraction in the L_∞ space while taking the argmin with respect to ϕ to update the model is a contraction in the L_2 space but the two together is not a contraction of any sort; thus, the loop that repeated sets the labels and takes the argmin with respect to ϕ may never converge.
4. We can technically think of the tabular variant as a version of the fitted variant where the model is just an input embedding vector of length $|S||A|$ and stores $Q(s, a)$.
5. We actually discard everything about the sample except for s_i when the transition dynamics $p(s'|s, a)$ are known in the fitted variant.
6. The fitted variant without knowledge of transition dynamics is possible because when we are taking the max over all actions, we just need to plug the action into the Q function instead of taking the action from that state in a simulator.

13 Model Free Fitted Q Iteration and Q-Learning

13.1 Basic Versions

13.1.1 Fitted Q Iteration

From before, in a model free setting where the transition dynamics are not known, fitted Q iteration is described by the following:

1.Repeat until Q_ϕ converges or for some steps M:

1a.Collect L random samples (s_i, a_i, s'_i, r_i) using any policy (possibly current policy with some exploration)

1b.Repeat until Q_ϕ converges or for some steps N:

1bi.For all samples :

$$1bi(1).y_i = r_i + \max_{a'} Q_\phi^\pi(s'_i, a')$$

$$1bii.\phi = \underset{\phi}{\operatorname{argmin}} \frac{1}{2} \sum_i \|Q_\phi(s_i, a_i) - y_i\|^2 \text{ (or run k steps of normal or stochastic gradient descent)}$$

By generalizing step 2b to take some steps of SGD, we get

1.Repeat until Q_ϕ converges or for some steps M:

1a.Collect L random samples (s_i, a_i, s'_i, r_i) using any policy (possibly current policy with some exploration)

1b.Repeat until Q_ϕ converges or for some steps N:

1bi.For all samples :

$$1bi(1).y_i = r_i + \max_{a'} Q_\phi^\pi(s'_i, a')$$

1bii.Repeat for batches (s_j, a_j, s'_j, r_j) from the random samples for some steps K :

$$1bii(1).\phi = \phi - \alpha \sum_j \frac{dQ_\phi}{d\phi}(s_j, a_j)(Q_\phi(s_j, a_j) - y_j)$$

13.1.2 Q Learning (Works for Online Setting)

1.Repeat until Q_ϕ converges or for some steps M:

1a.Collect one sample (s_i, a_i, s'_i, r_i) using any policy (possibly current policy with some exploration)

$$1b.y_i = r_i + \max_{a'} Q_\phi^\pi(s'_i, a')$$

$$1c.\phi = \phi - \alpha \frac{dQ_\phi}{d\phi}(s_i, a_i)(Q_\phi(s_i, a_i) - y_i)$$

13.1.3 Notes

1. Q Learning is the same as fitted Q iteration but $L = 1$ (we collect one sample at a time from the current trajectory and train on that sample before continuing), set $N = 1$ (do not keep resetting targets on same data once Q converges for current targets), and set $K = 1$ (only run one gradient descent step). Also, we fix the target for a samples before running SGD for fitted Q iteration while we reset the targets with the new parameters for Q learning between SGD (Q learning uses batch size 1) steps.

13.2 Variant With Replay Buffer

13.2.1 Fitted Q Iteration

1. Repeat until Q_ϕ converges or for some steps M:

1a. Add L random samples (s_i, a_i, s'_i, r_i) using any policy (possibly current policy with some exploration) replay buffer $\mathcal{B}$

1b. Repeat until Q_ϕ converges or for some steps N:

1bi. For all samples in replay buffer :

$$1bi(1).y_i = r_i + \max_{a'} Q_\phi^\pi(s'_i, a')$$

1bii. Repeat for batches (s_j, a_j, s'_j, r_j) from $\mathcal{B}$ for some steps K :

$$1bii(1).\phi = \phi - \alpha \sum_j \frac{dQ_\phi}{d\phi}(s_j, a_j)(Q_\phi(s_j, a_j) - y_j)$$

13.2.2 Q Learning (Works for Online Setting)

1. Repeat until Q_ϕ converges or for some steps M:

1a. Add one sample (s_i, a_i, s'_i, r_i) using any policy (possibly current policy with some exploration) to replay buffer $\mathcal{B}$

1b. Take one batch (s_j, a_j, s'_j, r_j) from $\mathcal{B}$

1bi. For every data point (s_j, a_j, s'_j, r_j) from the batch:

$$1bi(1).y_j = r_j + \max_{a'} Q_\phi^\pi(s'_j, a')$$

$$1bii.\phi = \phi - \alpha \sum_j \frac{dQ_\phi}{d\phi}(s_j, a_j)(Q_\phi(s_j, a_j) - y_j)$$

13.2.3 Notes

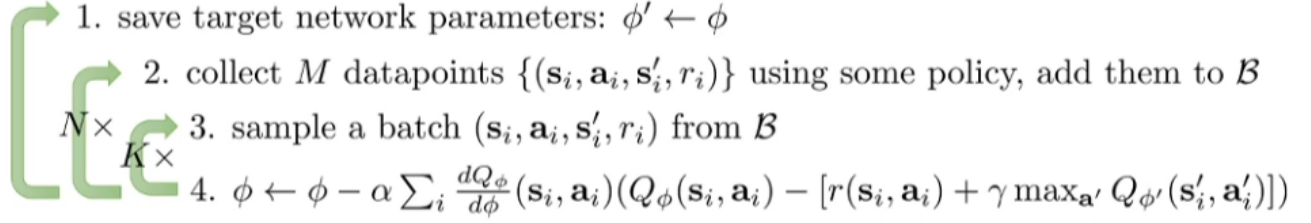
1. One concern with basic Q learning is that each sample is taken from one successive steps of the same trajectory (unless we are at the end of one trajectory and start over), which are highly correlated. This will cause the model to overfit to parts of a trajectory at a time. An example of where this can be bad is when we start a new trajectory, we just overfit to the end of the previous trajectory, which can cause the performance at the start of the new trajectory to be bad.

2. Fitted Q iteration does not suffer from the correlated samples issue since we train over random batches of samples even in the basic version, so the replay buffer here does not do much. The only difference with a replay buffer is that we do not discard the old data when collecting new samples in step 1a, so we take batches not just from the newly collect samples but from the entire replay buffer. Note that 1bi(1) can be achieved with a saved target network instead of storing y_i but the notation here is just for simplicity.

3. Same as before, Q Learning is the same as fitted Q iteration but $L = 1$ (we collect one sample at a time from the current trajectory and train on that sample before continuing), set $N = 1$ (do not keep resetting targets on same data once Q converges for current targets), and set $K = 1$ (only run one SGD step using one batch). Also, we fix the target for a samples before running SGD for fitted Q iteration while we reset the targets with the new parameters for Q learning between SGD (Q learning uses batch size 1) steps.

Fitted Q-iteration and Q-learning

Q-learning with replay buffer and target network:

- 
1. save target network parameters: $\phi' \leftarrow \phi$
 2. collect M datapoints $\{(s_i, a_i, s'_i, r_i)\}$ using some policy, add them to $\mathcal{B}$
 - $N \times K \times$ 3. sample a batch (s_i, a_i, s'_i, r_i) from $\mathcal{B}$
 4. $\phi \leftarrow \phi - \alpha \sum_i \frac{dQ_\phi}{d\phi}(s_i, a_i)(Q_\phi(s_i, a_i) - [r(s_i, a_i) + \gamma \max_{a'} Q_{\phi'}(s'_i, a'_i)])$

Fitted Q-learning (written similarly as above):

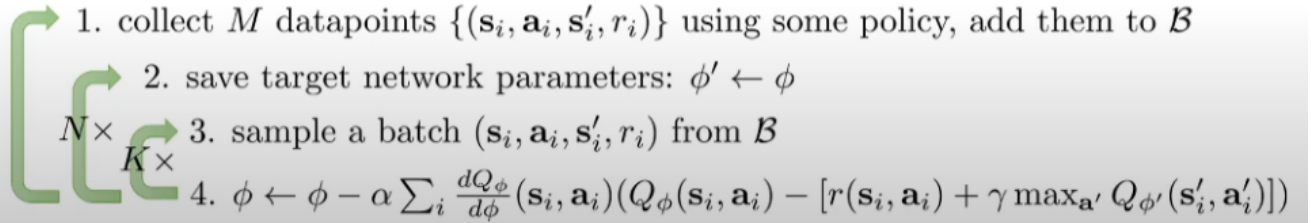
- 
1. collect M datapoints $\{(s_i, a_i, s'_i, r_i)\}$ using some policy, add them to $\mathcal{B}$
 2. save target network parameters: $\phi' \leftarrow \phi$
 - $N \times K \times$ 3. sample a batch (s_i, a_i, s'_i, r_i) from $\mathcal{B}$
 4. $\phi \leftarrow \phi - \alpha \sum_i \frac{dQ_\phi}{d\phi}(s_i, a_i)(Q_\phi(s_i, a_i) - [r(s_i, a_i) + \gamma \max_{a'} Q_{\phi'}(s'_i, a'_i)])$

Figure 1: Enter Caption

13.3 Variant With Replay Buffer and Target Network

13.3.1 Fitted Q Iteration

1.Repeat until Q_ϕ converges or for some steps M :

1a.Add L random samples (s_i, a_i, s'_i, r_i) using any policy (possibly current policy with some exploration) replay buffer $\mathcal{B}$

1b.Repeat until Q_ϕ converges or for some steps N :

1bi.Save target network parameters: $\phi' = \phi$

1bii.Repeat for different batches (s_j, a_j, s'_j, r_j) from $\mathcal{B}$ for some steps K :

1bii(1).For every data point (s_j, a_j, s'_j, r_j) from the batch:

1bii(1a). $y_j = r_j + \max_{a'} Q_{\phi'}^\pi(s'_j, a')$

1bii(2). $\phi = \phi - \alpha \sum_j \frac{dQ_\phi}{d\phi}(s_j, a_j)(Q_\phi(s_j, a_j) - y_j)$

13.3.2 Q Learning (Works for Online Setting) (DQN)

1.Repeat until Q_ϕ converges or for some steps M :

1a.Save target network parameters: $\phi' = \phi$

1b.Repeat for some steps N :

1bi.Add one sample (s_i, a_i, s'_i, r_i) using any policy (possibly current policy with some exploration) to replay buffer $\mathcal{B}$

1bii.Take one batch (s_j, a_j, s'_j, r_j) from $\mathcal{B}$:

1bii(1).For every data point (s_j, a_j, s'_j, r_j) from the batch:

1bii(1a). $y_j = r_j + \max_{a'} Q_{\phi'}^\pi(s'_j, a')$

1bii(2). $\phi = \phi - \alpha \sum_j \frac{dQ_\phi}{d\phi}(s_j, a_j)(Q_\phi(s_j, a_j) - y_j)$

13.3.3 Notes

1. Another concern for Q learning is that it has the issue with moving targets, making supervised learning hard to converge. Fitted Q iteration converges to targets, but the targets change after we finish converging with SGD, so converging to wrong targets may not be good either (e.g. more stability but wrong target values). Adding a target network to Q learning is a mix between the two.

2. Same as before, Q Learning is the same as fitted Q iteration but $L = 1$ (we collect one sample at a time from the current trajectory), set $K = 1$ only run one SGD step using one batch). However, we also do not have a moving target for Q learning. The main difference is that in fitted Q iteration, we train to convergence for the current targets for all existing data before updating the target network and gather data once Q has mostly converged for the current data while for Q learning we may or may not have trained to convergence for the current targets for all existing data (old data is likely converged but not likely for newer samples) and gather new data between SGD steps (do not wait for Q to converge on current data).

3. Note fitted Q iteration is the same with or without a target network.

14 Generalized Policy Improvement

The following is the proof for the policy improvement theorem:

Theorem 6. Policy Improvement Theorem: Consider two policy $\pi(a|s)$, $\pi'(a|s)$, and define

$$Q^\pi(s, \pi') = \mathbb{E}_{a \sim \pi'(a|s)}[Q^\pi(s, a)].$$

If $\forall s \in S$, we have that $Q^\pi(s, \pi') \geq V^\pi(s)$, then it holds that $V^{\pi'}(s) \geq V^\pi(s)$, $\forall s \in S$. This means that π' is atleast as good a policy as π .

Proof. Note that $Q^\pi(s, \pi') \geq V^\pi(s)$. By expanding Q^π , we can get that $\forall s \in S$,

$$\begin{aligned} V^\pi(s) &\leq Q^\pi(s, \pi') \\ &= \mathbb{E}_{a \sim \pi'(a|s), s' \sim \mathcal{T}'(s'|s, a)}[r(s, a) + \gamma V^\pi(s')] \\ &\leq \mathbb{E}_{a \sim \pi'(a|s), s' \sim \mathcal{T}'(s'|s, a)}[r(s, a) + \gamma Q^\pi(s', \pi')] \\ &= \mathbb{E}_{a, a' \sim \pi'}[r(s, a) + \gamma r(s', a') + \gamma^2 V^\pi(s'')] \\ &\leq \dots \\ &\leq \mathbb{E}_{a, a', a'' \dots \sim \pi'}[r(s, a) + \gamma r(s', a') + \gamma^2 r(s'', a'') + \dots] \\ &= V^{\pi'}(s) \end{aligned}$$

Figure 2: Policy Improvement Proof

Given n policies $\pi_1, \pi_2, \dots, \pi_n$ and their corresponding Q functions $Q^{\pi_1}, Q^{\pi_2}, \dots, Q^{\pi_n}$, let

$$\begin{aligned} Q^{\pi_k}(s, \pi'_k(s)) &= \max_a Q^{\pi_k}(s, a) \\ Q^{\pi_{k^*}(s)}(s, \pi'_{k^*}(s)) &= \max_a \max_k Q^{\pi_k}(s, a) = \max_k \max_a Q^{\pi_k}(s, a) \geq \max_a Q^{\pi_k}(s, a) \end{aligned}$$

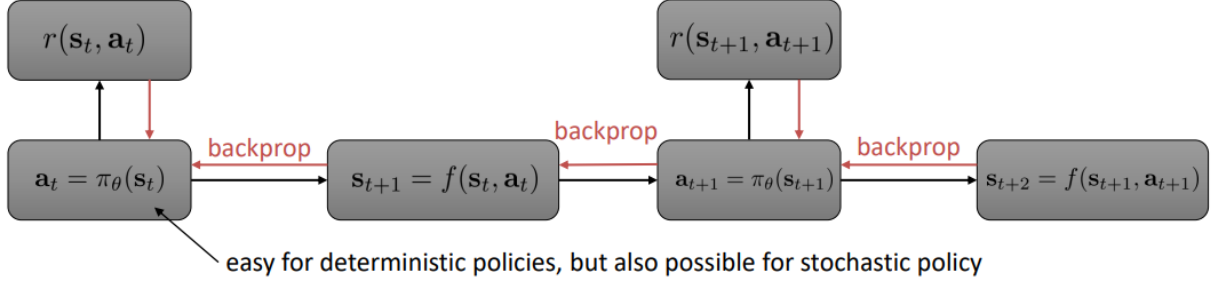
where $\pi'_k(s)$ represents the action a that maximizes $Q^{\pi_k}(s, a)$ for a given state s and $k^*(s)$ represents the policy at state s with the largest Q value assuming we are free to take any action from s . Also, we can define π^* where $\pi^*(a|s) = \pi_{k^*}(s)(a|s)$.

Using similar notation as the original proof, we get that for all policies k ,

$$\begin{aligned} V^{\pi_k}(s) &\leq Q^{\pi_k}(s, \pi'_k(s)) \\ &\leq Q^{\pi_{k^*}(s)}(s, \pi'_{k^*}(s)) \\ &= \mathbb{E}_{a \sim \pi'_{k^*}(s)}(a|s), s' \sim T(s, a) [r(s, a) + \gamma V^{\pi_{k^*}(s)}(s')|s] \\ &= \mathbb{E}_{\tau \sim p_{\pi_{k^*}}(\tau)} [r(s, a) + \gamma V^{\pi_{k^*}(s)}(s')|s] \\ &\leq \mathbb{E}_{\tau \sim p_{\pi_{k^*}}(\tau)} [r(s, a) + \gamma Q^{\pi_{k^*}(s)}(s', \pi'_{k^*}(s'))|s] \\ &\leq \mathbb{E}_{\tau \sim p_{\pi_{k^*}}(\tau)} [r(s, a) + \gamma Q^{\pi_{k^*}(s')}(s', \pi'_{k^*}(s'))|s] \\ &= \mathbb{E}_{\tau \sim p_{\pi_{k^*}}(\tau)} \left[r(s, a) + \gamma \mathbb{E}_{a' \sim \pi'_{k^*}(s')}(a'|s'), s'' \sim T(s', a') [r(s', a') + \gamma V^{\pi_{k^*}(s')}(s'')|s'|s] \right] \\ &= \mathbb{E}_{\tau \sim p_{\pi_{k^*}}(\tau)} \left[r(s, a) + \gamma \mathbb{E}_{\tau' \sim p_{\pi_{k^*}}(\tau')} [r(s', a') + \gamma V^{\pi_{k^*}(s')}(s'')|s, s']|s \right] \\ &= \mathbb{E}_{\tau \sim p_{\pi_{k^*}}(\tau)} [r(s, a) + \gamma r(s', a') + \gamma^2 V^{\pi_{k^*}(s')}(s'')|s] \\ &\leq \dots \\ &\leq \mathbb{E}_{\tau \sim p_{\pi_{k^*}}(\tau)} [r(s, a) + \gamma r(s', a') + \gamma^2 r(s'', a'') + \dots |s] \\ &= V^{\pi^*}(s) \end{aligned}$$

Note we used the Markov property first and then applied the tower rule.

15 Model Based RL Policy Learning



model-based reinforcement learning version 2.0:

1. run base policy $\pi_0(\mathbf{a}_t|\mathbf{s}_t)$ (e.g., random policy) to collect $\mathcal{D} = \{(\mathbf{s}, \mathbf{a}, \mathbf{s}')_i\}$
2. learn dynamics model $f(\mathbf{s}, \mathbf{a})$ to minimize $\sum_i \|f(\mathbf{s}_i, \mathbf{a}_i) - \mathbf{s}'_i\|^2$
3. backpropagate through $f(\mathbf{s}, \mathbf{a})$ into the policy to optimize $\pi_\theta(\mathbf{a}_t|\mathbf{s}_t)$
4. run $\pi_\theta(\mathbf{a}_t|\mathbf{s}_t)$, appending the visited tuples $(\mathbf{s}, \mathbf{a}, \mathbf{s}')$ to $\mathcal{D}$

Figure 3: Back Propagation Through Dynamics Model

Given a dynamics model produced by a neural network, we can backpropagate the reward function directly through the dynamics model backwards in time for each action to update the policy parameters as follows (assuming the reward is only dependent on the state, which differs slightly from the diagram that has a reward dependent on both state and action):

$$\begin{aligned}
 \nabla_\theta J(\theta) &= \sum_{t=1}^T \frac{da_t}{d\theta} \frac{ds_{t+1}}{da_t} \left(\sum_{t'=t+1}^T \frac{dr_{t'}}{ds_{t'}} \left(\prod_{t''=t+2}^{t'} \frac{ds_{t''}}{da_{t''-1}} \frac{da_{t''-1}}{ds_{t''-1}} + \frac{ds_{t''}}{ds_{t''-1}} \right) \right) \\
 &= \sum_{t=1}^T \frac{dr_t}{ds_t} \left(\sum_{t'=1}^{t-1} \frac{ds_{t'+1}}{da_{t'}} \frac{da_t}{d\theta} \prod_{t''=t'+2}^t \left(\frac{ds_{t''}}{da_{t''-1}} \frac{da_{t''-1}}{ds_{t''-1}} + \frac{ds_{t''}}{ds_{t''-1}} \right) \right)
 \end{aligned}$$

where we can think of the first equation as looking at the each at each time step and summing all gradients from future rewards. The second equation can be thought of as looking at each reward and backpropagating the reward to all past actions. Note that each $s_{t+1} = f(s_t, a_t)$, so

$$\frac{ds_{t+1}}{ds_t} = \frac{d}{ds_t} f(s_t, a_t) + \frac{d}{da_t} f(s_t, a_t) \frac{da_t}{ds_t}$$

which is exactly the term in the innermost product.

Note that this is similar to backpropagation through time.

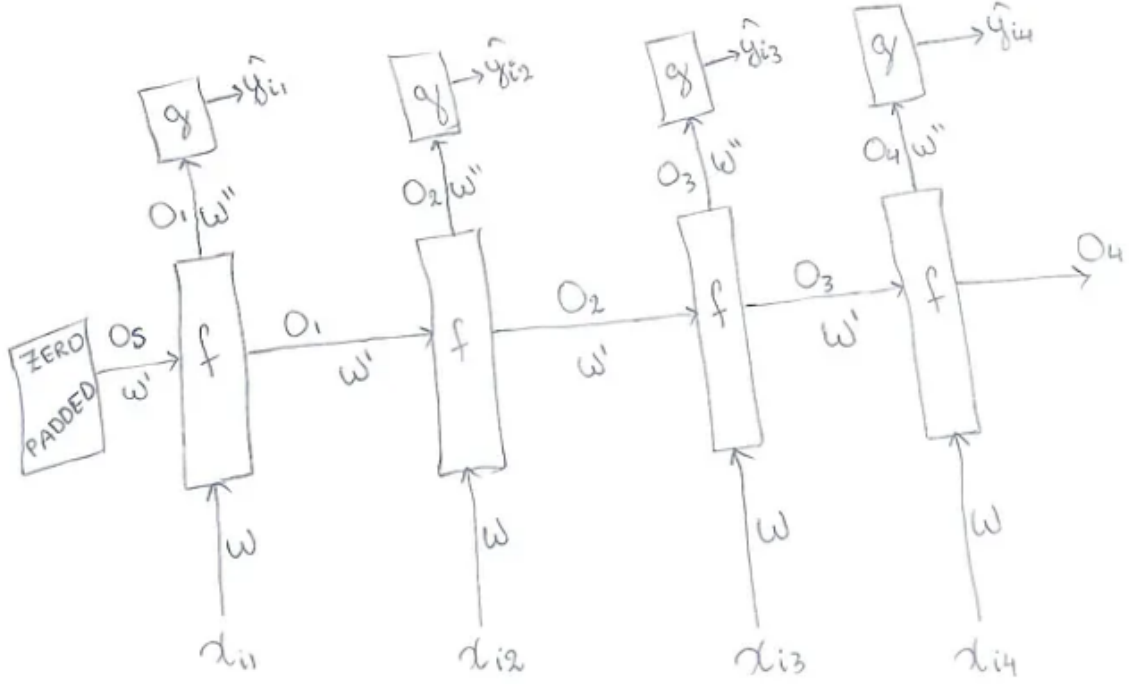


Figure 4: RNN

We can subscript ω , ω' , and ω'' to represent their corresponding values at each time step (S corresponds to 0). The values at different time steps are the same, but we are doing this since the overall gradient with respect to ω , ω' , and ω'' are the sum of the gradients with respect to ω , ω' , and ω'' at each individual time step. Using the substitution $z_t = (\omega_t x_t + \omega'_t o_{t-1})$ and $v_t = \omega''_t o_t$:

$$\begin{aligned}\nabla_{\omega'} L &= \sum_{t=1}^T \frac{dz_t}{d\omega'_t} \frac{do_t}{dz_t} \sum_{t'=t}^T \frac{dL_{t'}}{d\hat{y}_{t'}} \frac{d\hat{y}_{t'}}{dv_{t'}} \frac{dv_{t'}}{do_{t'}} \left(\prod_{t''=t+1}^{t'} \frac{do_{t''}}{dz_{t''}} \frac{dz_{t''}}{do_{t''-1}} \right) \\ &= \sum_{t=1}^T \frac{dL_t}{d\hat{y}_t} \frac{d\hat{y}_t}{dv_t} \frac{dv_t}{do_t} \sum_{t'=1}^t \frac{dz_{t'}}{d\omega'_t} \frac{do_{t'}}{dz_{t'}} \left(\prod_{t''=t'+1}^t \frac{do_{t''}}{dz_{t''}} \frac{dz_{t''}}{do_{t''-1}} \right)\end{aligned}$$

Similarly

$$\begin{aligned}\nabla_{\omega} L &= \sum_{t=1}^T \frac{dz_t}{d\omega_t} \frac{do_t}{dz_t} \sum_{t'=t}^T \frac{dL_{t'}}{d\hat{y}_{t'}} \frac{d\hat{y}_{t'}}{dv_{t'}} \frac{dv_{t'}}{do_{t'}} \left(\prod_{t''=t+1}^{t'} \frac{do_{t''}}{dz_{t''}} \frac{dz_{t''}}{do_{t''-1}} \right) \\ &= \sum_{t=1}^T \frac{dL_t}{d\hat{y}_t} \frac{d\hat{y}_t}{dv_t} \frac{dv_t}{do_t} \sum_{t'=1}^t \frac{dz_{t'}}{d\omega_t} \frac{do_{t'}}{dz_{t'}} \left(\prod_{t''=t'+1}^t \frac{do_{t''}}{dz_{t''}} \frac{dz_{t''}}{do_{t''-1}} \right)\end{aligned}$$

and

$$\nabla_{\omega''} L = \sum_{t=1}^T \frac{dL_t}{d\hat{y}_t} \frac{d\hat{y}_t}{dv_t} \frac{dv_t}{d\omega''_t}$$

$O_i = f(wx_{ii} + w'O_s)$
 $O_i = f(wx_{ii})$
 $\hat{y}_{ii} = g(w'O_i)$

$w'O_s \rightarrow$ is a zero because it being zero-padded longer

Figure 5: Foward Prop Equations

16 Successor Features

$$\begin{aligned}
V^\pi(s_t) &= \mathbb{E}_{a_t, \tau_{t+1:\infty} \sim p_\pi(\tau)} \left[\sum_{t'=t}^{\infty} \gamma^{t'-t} r(s_{t'}, a_{t'}) \middle| s_t \right] \\
&= \sum_{t'=t}^{\infty} \gamma^{t'-t} \mathbb{E}_{s_{t'} \sim p_\pi(s_{t'}|s_t), a_{t'} \sim \pi(a_{t'}|s_{t'})} [r(s_{t'}, a_{t'}) | s_t] \\
&= \sum_{t'=t}^{\infty} \sum_s \sum_a \gamma^{t'-t} p_\pi(s_{t'} = s | s_t) \pi(a | s_{t'}) r(s, a) \\
&= \sum_s \sum_a r(s, a) \left(\sum_{t'=t}^{\infty} \gamma^{t'-t} p_\pi(s_{t'} = s | s_t) \pi(a | s_{t'}) \right)
\end{aligned}$$

Note that for any fixed timestep t'

$$\sum_s \sum_a p_\pi(s_{t'} = s | s_t) \pi(a | s_{t'}) = 1$$

Thus,

$$\begin{aligned}
\sum_s \sum_a \sum_{t'=t}^{\infty} \gamma^{t'-t} p_\pi(s_{t'} = s | s_t) \pi(a | s_{t'}) &= \sum_{t'=t}^{\infty} \gamma^{t'-t} \sum_s \sum_a p_\pi(s_{t'} = s | s_t) \pi(a | s_{t'}) \\
&= \sum_{t'=t}^{\infty} \gamma^{t'-t} \\
&= \frac{1}{1 - \gamma}
\end{aligned}$$

Thus,

$$p_\pi(s_{future} = s, a_{future} = a | s_t) = (1 - \gamma) \sum_{t'=t}^{\infty} \gamma^{t'-t} p_\pi(s_{t'} = s | s_t) \pi(a | s_{t'})$$

is a distribution over (s, a) since

$$\sum_s \sum_a (1 - \gamma) \sum_{t'=t}^{\infty} \gamma^{t'-t} p_\pi(s_{t'} = s | s_t) \pi(a | s_{t'}) = 1$$

More specifically, we can define a process where we start in state s_t and take action a_t . At every time step, we have a $(1 - \gamma)$ probability of terminating after entering the state and taking an action (including the first state and

action s_t, a_t). Then $p(s_{future} = s, a_{future} = a | s_t)$ represents the probability we terminate after entering state s and taking the action a . Substituting this probability expression into $V^\pi(s_t)$, we get

$$\begin{aligned}
V^\pi(s_t) &= \mathbb{E}_{a_t, \tau_{t+1}:\infty \sim p_\pi(\tau)} \left[\sum_{t'=t}^{\infty} \gamma^{t'-t} r(s_{t'}, a_{t'}) \middle| s_t \right] \\
&= \sum_s \sum_a r(s, a) \left(\sum_{t'=t}^{\infty} \gamma^{t'-t} p_\pi(s_{t'} = s | s_t) \pi(a | s_{t'}) \right) \\
&= \frac{1}{1-\gamma} \sum_s \sum_a r(s, a) \left((1-\gamma) \sum_{t'=t}^{\infty} \gamma^{t'-t} p_\pi(s_{t'} = s | s_t) \pi(a | s_{t'}) \right) \\
&= \frac{1}{1-\gamma} \sum_s \sum_a r(s, a) p_\pi(s_{future} = s, a_{future} = a | s_t) \\
&= \frac{1}{1-\gamma} \mu^\pi(s_t)^T \vec{r}
\end{aligned}$$

where $\vec{r}$ and $\mu^\pi(s_t)$ are vectors of length $|S||A|$ that store $r(s, a)$ and $p(s_{future} = s, a_{future} = a | s_t)$ respectively as elements for all possible (s, a) pairs. Similarly

$$\begin{aligned}
Q^\pi(s_t, a_t) &= \mathbb{E}_{a_t, \tau_{t+1}:\infty \sim p_\pi(\tau)} \left[\sum_{t'=t}^{\infty} \gamma^{t'-t} r(s_{t'}, a_{t'}) \middle| s_t, a_t \right] \\
&= \sum_{t'=t}^{\infty} \gamma^{t'-t} \mathbb{E}_{s_{t'} \sim p_\pi(s_{t'} | s_t, a_t), a_{t'} \sim \pi(a_{t'} | s_{t'})} [r(s_{t'}, a_{t'}) | s_t, a_t] \\
&= \sum_{t'=t}^{\infty} \sum_s \sum_a \gamma^{t'-t} p_\pi(s_{t'} = s | s_t, a_t) \pi(a | s_{t'}) r(s, a) \\
&= \frac{1}{1-\gamma} \sum_s \sum_a r(s, a) \left((1-\gamma) \sum_{t'=t}^{\infty} \gamma^{t'-t} p_\pi(s_{t'} = s | s_t, a_t) \pi(a | s_{t'}) \right) \\
&= \frac{1}{1-\gamma} \sum_s \sum_a r(s, a) p_\pi(s_{future} = s, a_{future} = a | s_t, a_t) \\
&= \frac{1}{1-\gamma} \mu^\pi(s_t, a_t)^T \vec{r}
\end{aligned}$$

where $\vec{r}$ and $\mu^\pi(s_t, a_t)$ are vectors of length $|S||A|$ that store $r(s, a)$ and $p(s_{future} = s, a_{future} = a | s_t, a_t)$ as elements for all possible (s, a) pairs. Similar to before, we can define a process where we start in state s_t and take action a_t . At every time step, we have a $(1-\gamma)$ probability of terminating after entering the state and taking an action (including the first state and action s_t, a_t). Then $p_\pi(s_{future} = s, a_{future} = a | s_t)$ represents the probability terminate after landing in state s and taking the action a .

Next, note that we can express $\mu_{i,j}^\pi(s_t, a_t)$ (the element of vector $\mu(s_t, a_t)$ that corresponds with state i and action j) recursively with a Bellman equation:

$$\mu_{i,j}^\pi(s_t, a_t) = (1-\gamma)\delta(s_t = i, a_t = j) + \gamma \mathbb{E}_{a_{t+1} \sim \pi(a_{t+1} | s_{t+1}), s_{t+1} \sim p(s_{t+1} | s_t, a_t)} [\mu_{i,j}^\pi(s_{t+1}, a_{t+1})]$$

This is basically a first step equation (like for Markov chains) where the first term represents the probability of terminating in state i after taking action j if we terminate after the first action in timestep t and the second equation represents the probability of terminating in state i after taking action j if we do not terminate after the first action in timestep t .

Note that $\mu^\pi(s_t, a_t)$ depends on the policy but not the reward, so if we solve for $\mu^\pi(s_t, a_t)$ and want to solve a similar MDP where everything is the same except for the reward function, we can easily compute $Q^\pi(s_t, a_t)$ given the new reward, allowing for transfer learning

However, the main downside of this is that we have to learn $\mu^\pi(s_t, a_t)$ for all possible s_t, a_t pairs. We can mitigate

this if the set of reward functions can be well approximated with a linear combination of features ϕ^π , which are essentially basis vectors for r . Assuming, this is the case, lets define the following:

$$\begin{aligned}\vec{r} &= \phi^\pi \vec{w} \\ \text{where } \phi &= [\phi_1^\pi \quad \phi_2^\pi \quad \dots \quad \phi_n^\pi] \\ r(s, a) &= \sum_j \phi_j^\pi(s, a) w_j = \phi^\pi(s, a)^T \vec{w}\end{aligned}$$

Note that ϕ^π is a $|S||A|$ by n matrix and $\phi^\pi(s, a)$ is the row of ϕ corresponding to state s and action a .

We can think of each ϕ_j^π as a psuedo reward, that generates a psuedo Q (or value) function ψ_j^π , that is like a basis for the Q (or value) function:

$$\begin{aligned}\psi^\pi(s, a) &= \begin{bmatrix} \psi_1^\pi(s, a) \\ \psi_2^\pi(s, a) \\ \vdots \\ \psi_n^\pi(s, a) \end{bmatrix} = \phi^{\pi T} \mu^\pi(s_t, a_t) \\ \psi_j^\pi(s_t, a_t) &= \frac{1}{1-\gamma} \mu^\pi(s_t, a_t)^T \phi_j^\pi \\ Q^\pi(s_t, a_t) &= \frac{1}{1-\gamma} \mu^\pi(s_t, a_t)^T \vec{r} = \mu^\pi(s_t, a_t)^T \phi^\pi \vec{w} = (\phi^{\pi T} \mu^\pi(s_t, a_t))^T \vec{w} = \psi^\pi(s_t, a_t)^T \vec{w}\end{aligned}$$

Note that $\psi_j^\pi(s, a)$ is just the $Q^\pi(s, a)$ if the reward was defined by ϕ_j^π ; thus, we can write a similar recursive Bellman equation for each $\psi_j(s, a)$:

$$\psi_j^\pi(s_t, a_t) = \phi_j^\pi(s_t, a_t) + \gamma \mathbb{E}_{s_{t+1} \sim p(s_{t+1}|s_t, a_t), a_{t+1} \sim \pi(a_{t+1}|s_{t+1})} [\psi_j^\pi(s_{t+1}, a_{t+1})]$$

Given a basis ϕ^π we can now learn ψ^π by repeated performing Bellman backups. Once we learn ψ , we can easily recover $Q^\pi(s, a)$ for any reward function given by $\vec{w}$ with $\psi^\pi(s_t, a_t)^T \vec{w}$.

Note that when have one basis vector $\phi_{i,j}$ for every state action pair (s, a) that is 1 for the element corresponding to state action pair enumerated by j and zero everywhere else (e.g. $\phi_{i,j}(s_t, a_t) = \delta(s_t = i, a_t = j)$ and ϕ is a $|S||A|$ by $|S||A|$ identity matrix), we get that each successor feature $\psi_{i,j} = \mu_{i,j}$ and $\vec{w} = \vec{r}$.

Once, we have learned ψ can use it for transfer learning by easily computing Q^{π_i} for any policies $\pi_1, \pi_2, \dots, \pi_n$. And when we want to learn a good policy for a new reward, we can run generalized policy improvement to get a new policy π' such that has a $Q^{\pi'}(s, a) \geq \max_i Q^{\pi_i}(s, a)$ (policy is is at least as good as the old policies).

For learning the classifier $p^\pi(F = 1|s_t, a_t, s_{future})$ for contiuous state spaces, we are assuming equal priors with $p^\pi(F = 1) = p^\pi(F = 0) = \frac{1}{2}$. Note that sampling a state s by selecting a random (s, a) pair uniformly from the distribution of trajectories and sampling from $p^\pi(s_{future}|s_t, a_t)$ is about the same as just sampling s from the distribution of trajectories when trajectories are long and s is close to the end of the trajectory. Imagine we only have one deterministic trajectory of length $|S|$ (assume it is discrete to simplify the example) that visits every state once.

$$p^\pi(s_t) = \frac{1}{|S|}$$

but

$$\sum_{t'=1}^t p^\pi(s_t|s_{t'}, a_{t'}) p^\pi(s_{t'}, a_{t'}) = \frac{1}{|S|} \sum_{t'=1}^t p^\pi(s_t|s_{t'}, a_{t'}) = \frac{1}{|S|} \sum_{t'=1}^t (1-\gamma) \gamma^{t'-t}$$

Note that $\sum_{t'=1}^t (1-\gamma) \gamma^{t'-t} = 1$ only if $t = \infty$. Also, it is likely that the $F = 0$ distribution $p^\pi(s)$ does not need to be the one defined by

$$p^\pi(s) = \sum_t p^\pi(s|s_t, a_t) p^\pi(s_t, a_t)$$

but any distribution (including something random like a uniform or Gaussian) as long as it only depends on the state s and not the action (so $p^\pi(s)$ does not affect the max over actions) in the expression

$$\frac{p^\pi(F = 1|s_t, a_t, s_{future})}{p^\pi(F = 0|s_t, a_t, s_{future})} = \frac{p^\pi(s_{future}|s_t, a_t)}{p^\pi(s_{future})}$$

17 Exploration

Thompson sampling- Each state action pair (s, a) is a bandit and $Q(s, a)$ is the reward. We sample a single Q function for all state action pairs and act accordingly.

18 Miscellaneous

Another way to prove lemma: Aperiodic means that there is a timestep t such that $P(x, x) > 0$ for both t and $t + 1$. Using Extended Euclid, we know $c_1 t_1 + c_2 t_2 + \dots c_n t_n = 1$ for some $t_1, t_2, \dots, t_n \in T(x)$ and

$$c_2 t_2 + \dots c_n t_n \equiv 1 \pmod{t_1}$$

Let $\tilde{t}_1, \tilde{t}_2, \dots, \tilde{t}_n$ be the respective least positive residue of $t_2, t_3, \dots, t_n$. We know that

$$c_2 \tilde{t}_2 + \dots c_n \tilde{t}_n = kt_1 + 1 \text{ for some } k \in \mathbb{N}$$

Thus, we know that $P(x, x) > 0$ for both $kt_1 + 1$ and kt_1 . Similarly, we can continue this reasoning to see that $P(x, x) > 0$ for $2k_t = kt_1 + kt_1$, $2k_t + 1 = kt_1 + kt_1 + 1$, $2k_t + 2 = kt_1 + 1 + kt_1 + 1$. Continue by induction.

1.3. Irreducibility and Aperiodicity

We now make note of two simple properties possessed by most interesting chains. Both will turn out to be necessary for the Convergence Theorem (Theorem 4.9) to be true.

A chain P is called **irreducible** if for any two states $x, y \in \Omega$ there exists an integer t (possibly depending on x and y) such that $P^t(x, y) > 0$. This means that it is possible to get from any state to any other state using only transitions of positive probability. We will generally assume that the chains under discussion are irreducible. (Checking that specific chains are irreducible can be quite interesting; see, for instance, Section 2.6 and Example B.5. See Section 1.7 for a discussion of all the ways in which a Markov chain can fail to be irreducible.)

Let $\mathcal{T}(x) := \{t \geq 1 : P^t(x, x) > 0\}$ be the set of times when it is possible for the chain to return to starting position x . The **period** of state x is defined to be the greatest common divisor of $\mathcal{T}(x)$.

LEMMA 1.6. *If P is irreducible, then $\gcd \mathcal{T}(x) = \gcd \mathcal{T}(y)$ for all $x, y \in \Omega$.*

PROOF. Fix two states x and y . There exist non-negative integers r and ℓ such that $P^r(x, y) > 0$ and $P^\ell(y, x) > 0$. Letting $m = r + \ell$, we have $m \in \mathcal{T}(x) \cap \mathcal{T}(y)$ and $\mathcal{T}(x) \subset \mathcal{T}(y) - m$, whence $\gcd \mathcal{T}(y)$ divides all elements of $\mathcal{T}(x)$. We conclude that $\gcd \mathcal{T}(y) \leq \gcd \mathcal{T}(x)$. By an entirely parallel argument, $\gcd \mathcal{T}(x) \leq \gcd \mathcal{T}(y)$. ■

For an irreducible chain, the period of the chain is defined to be the period which is common to all states. The chain will be called **aperiodic** if all states have period 1. If a chain is not aperiodic, we call it **periodic**.

PROPOSITION 1.7. *If P is aperiodic and irreducible, then there is an integer r such that $P^r(x, y) > 0$ for all $x, y \in \Omega$.*

PROOF. We use the following number-theoretic fact: any set of non-negative integers which is closed under addition and which has greatest common divisor 1 must contain all but finitely many of the non-negative integers. (See Lemma 1.27 in the Notes of this chapter for a proof.) For $x \in \Omega$, recall that $\mathcal{T}(x) = \{t \geq 1 : P^t(x, x) > 0\}$. Since the chain is aperiodic, the gcd of $\mathcal{T}(x)$ is 1. The set $\mathcal{T}(x)$ is closed under addition: if $s, t \in \mathcal{T}(x)$, then $P^{s+t}(x, x) \geq P^s(x, x)P^t(x, x) > 0$, and hence $s + t \in \mathcal{T}(x)$. Therefore there exists a $t(x)$ such that $t \geq t(x)$ implies $t \in \mathcal{T}(x)$. By irreducibility we know that for any $y \in \Omega$ there exists $r = r(x, y)$ such that $P^r(x, y) > 0$. Therefore, for $t \geq t(x) + r$,

$$P^t(x, y) \geq P^{t-r}(x, x)P^r(x, y) > 0.$$

For $t \geq t'(x) := t(x) + \max_{y \in \Omega} r(x, y)$, we have $P^t(x, y) > 0$ for all $y \in \Omega$. Finally, if $t \geq \max_{x \in \Omega} t'(x)$, then $P^t(x, y) > 0$ for all $x, y \in \Omega$. ■

Figure 6: Irreducible Markov Chains

LEMMA 1.27. *If $S \subset \mathbb{Z}^+$ has $\gcd(S) = g_S$, then there is some integer m_S such that for all $m \geq m_S$ the product mg_S can be written as a linear combination of elements of S with non-negative integer coefficients.*

PROOF. *Step 1.* Given $S \subset \mathbb{Z}^+$ nonempty, define g_S^* as the smallest positive integer which is an integer combination of elements of S (the smallest positive element of the additive group generated by S). Then g_S^* divides every element of S (otherwise, consider the remainder) and g_S must divide g_S^* , so $g_S^* = g_S$.

Step 2. For any set S of positive integers, there is a finite subset F such that $\gcd(S) = \gcd(F)$. Indeed the non-increasing sequence $\gcd(S \cap [1, n])$ can strictly decrease only finitely many times, so there is a last time. Thus it suffices to prove the fact for finite subsets F of $\mathbb{Z}^+$; we start with sets of size 2 (size 1 is a tautology) and then prove the general case by induction on the size of F .

Step 3. Let $F = \{a, b\} \subset \mathbb{Z}^+$ have $\gcd(F) = g$. Given $m > 0$, write $mg = ca + db$ for some integers c, d . Observe that c, d are not unique since $mg = (c + kb)a + (d - ka)b$ for any k . Thus we can write $mg = ca + db$ where $0 \leq c < b$. If $mg > (b - 1)a - b$, then we must have $d \geq 0$ as well. Thus for $F = \{a, b\}$ we can take $m_F = (ab - a - b)/g + 1$.

Step 4 (The induction step). Let F be a finite subset of $\mathbb{Z}^+$ with $\gcd(F) = g_F$. Then for any $a \in \mathbb{Z}^+$ the definition of $\gcd$ yields that $g := \gcd(\{a\} \cup F) = \gcd(a, g_F)$. Suppose that n satisfies $ng \geq m_{\{a, g_F\}}g + m_F g_F$. Then we can write $ng - m_F g_F = ca + dg_F$ for integers $c, d \geq 0$. Therefore $ng = ca + (d + m_F)g_F = ca + \sum_{f \in F} c_f f$ for some integers $c_f \geq 0$ by the definition of m_F . Thus we can take $m_{\{a\} \cup F} = m_{\{a, g_F\}} + m_F g_F / g$. ■

Figure 7: Lemma

19 Entropy Exploration

When $\alpha = -1$, we are just maximizing the entropy where the real distribution is a uniform (scaled by some constant). Note that the most recent policy avoids all past visited states, but the learned prior over all visited states provides a good density estimate of all states seen so far (if we only learned a prior over the most recent rollout, we would only get a density estimate for the last rollout). And to make the final policy also have good coverage, we need to return a mixture model that averages all policies seen so far since all of the policies together produced the states seen so far, not just the last one. The last policy only avoids previously seen states, but we want a policy that visits all states. This is similar to Skew Fit, except the reward of maximizing the probability of classifying the correct task is less sparse (not just whether we reached the goal or not). This goal of being able to accurately estimate the task based on the state, means different tasks visit different states. Skew Fit maximizes mutual information between the final state and the goal while learning diverse skills, for each state, maximizes the mutual information between the task and the state.

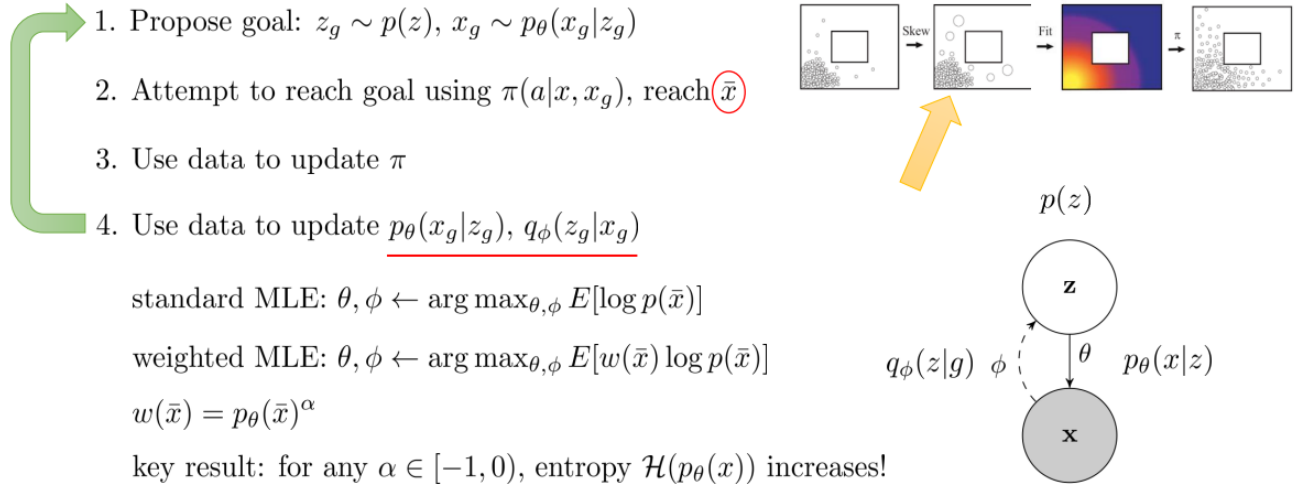


Figure 8: SkewFit

the state marginal matching problem: learn $\pi(\mathbf{a}|\mathbf{s})$ so as to minimize $D_{\text{KL}}(p_\pi(\mathbf{s})||p^*(\mathbf{s}))$

idea: can we use intrinsic motivation?

$$\tilde{r}(\mathbf{s}) = \log p^*(\mathbf{s}) - \log p_\pi(\mathbf{s})$$

this does **not** perform marginal matching!

1. learn $\pi^k(\mathbf{a}|\mathbf{s})$ to maximize $E_\pi[\tilde{r}^k(\mathbf{s})]$
- ~~2. update $p_{\pi^k}(\mathbf{s})$ to fit state marginal~~
2. update $p_{\pi^k}(\mathbf{s})$ to fit *all states seen so far*
3. return $\pi^*(\mathbf{a}|\mathbf{s}) = \sum_k \pi^k(\mathbf{a}|\mathbf{s})$

this **does** perform marginal matching!

$p_\pi(\mathbf{s}) = p^*(\mathbf{s})$ is Nash equilibrium of two player game between π^k and p_{π^k}

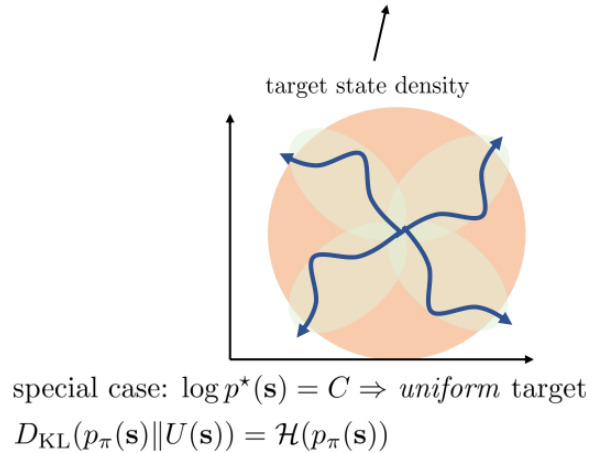


Figure 9: State Marginal Matching

$$\pi(\mathbf{a}|\mathbf{s}, z) = \arg \max_{\pi} \sum_z E_{\mathbf{s} \sim \pi(\mathbf{s}|z)} [r(\mathbf{s}, z)]$$

↑
reward states that are unlikely for other $z' \neq z$

$$r(\mathbf{s}, z) = \log p(z|\mathbf{s})$$

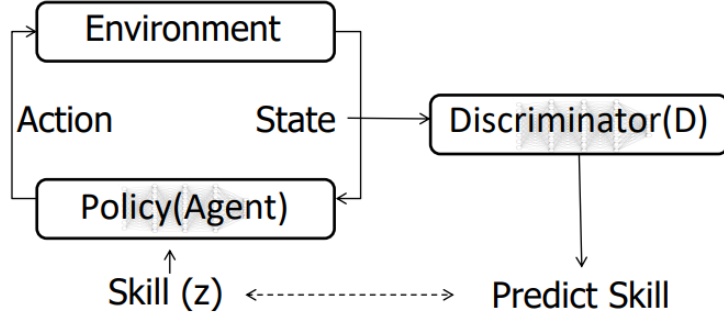
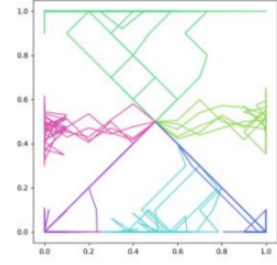
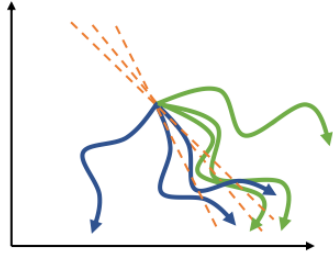


Figure 10: Learning Diverse Skills

20 Transfer Learning

In transfer learning, the goal is to use "experience from learning one set of tasks for faster learning and better performance on a new task." Essentially, we train a model over a source domain or domains and try to apply that model in a different target domain, where a domain can be thought of as a data distribution.

20.1 Domain Shift

One such issue that can arise in transfer learning is domain (e.g. distribution) shift, where the representations learned by the network for the source domain do not work well in the target domain. The distance between domains D and D' can be measured using L1 distance (e.g. 2x the TVD), which can be expressed as

$$d_1(D, D') = 2 \sup_{B \in \mathcal{B}} |P_D[B] - P_{D'}[B]|$$

However, this is often intractable to estimate in practice, and we instead opt for a $\mathcal{H}$ divergence, where each $h \in \mathcal{H}$ is a characteristic (e.g. indicator) function mapping samples from the domain to $\{0, 1\}$

$$d_{\mathcal{H}}(D, D') = 2 \sup_{h \in \mathcal{H}} |P_D[h] - P_{D'}[h]|$$

where $I(h) = \{x : h(x) = 1\}$. If $\mathcal{H}$ is symmetric (e.g. $h \in \mathcal{H} \implies (1 - h) \in \mathcal{H}$)

$$\begin{aligned}
& P_D[I(h)] - P_{D'}[I(h)] \\
&= \frac{1}{2}(P_D[I(h)] - P_{D'}[I(h)]) + \frac{1}{2}(P_D[I(h)] - P_{D'}[I(h)]) \\
&= \frac{1}{2}(1 - P_{D'}[I(h)] - (1 - P_D[I(h)])) + \frac{1}{2}(P_D[I(h)] - P_{D'}[I(h)]) \\
&= \frac{1}{2}(1 - P_{D'}[I(h)] + (1 - P_D[I(h)]) - 2(1 - P_D[I(h)])) + \frac{1}{2}(P_D[I(h)] + P_{D'}[I(h)] - 2P_{D'}[I(h)]) \\
&= \frac{1}{2}(1 - P_{D'}[I(h)] + (1 - P_D[I(h)])) + \frac{1}{2}(P_D[I(h)] + P_{D'}[I(h)] - (1 - P_D[I(h)]) - P_{D'}[I(h)]) \\
&= \frac{1}{2}(1 - P_{D'}[I(h)] + (1 - P_D[I(h)]) + P_D[I(h)] + P_{D'}[I(h)] - (1 - P_D[I(h)]) - P_{D'}[I(h)]) \\
&= \frac{1}{2}(2) - (1 - P_D[I(h)]) - P_{D'}[I(h)] \\
&= 1 - (1 - P_D[I(h)]) - P_{D'}[I(h)] \\
&= 1 - ((1 - P_D[I(h)]) + P_{D'}[I(h)])
\end{aligned}$$

Thus, maximizing hypothesis $|P_D[I(h)] - P_{D'}[I(h)]|$ is the one that minimizes $(1 - P_D[I(h)]) + P_{D'}[I(h)]$, which can be interpreted the sum of the probability of misclassifying a point from D as a point from D' and the probability of misclassifying a point from D' as point from D . This can be done by training a discriminator.

To mitigate the problem of domain shift, we want $\pi_t \text{heta}(a|s)$ to have a feature representation of s such that $d_{\mathcal{H}}(D, D')$ is small as possible, where D is the distribution of feature representations from the source domain and D' is the distribution of feature representations from the target domain. Thus, we can try our policy as usual but add an extra head to the network on top of a hidden layer, which we will set as the feature representation. from the policy network. This extra head will be a discriminator that predicts 1 for target domain feature representations and 0 for source domain feature representations (or vice versa). This extra head can be trained with a GAN loss or just logistic loss. However, we only back propagate the gradient of this discriminator loss through discriminator and multiply the loss by -1 when propagating it to the shared network that is used for policy predictions since we want the shared part of the network to produce feature representations between the source and target domain of inputs to be hard to distinguish for the discriminator.

20.2 Differences in MDP

Other times, there are things we can do in the source domain but not the target domain and vice versa. To mitigate the problem of learning actions that take advantage of transitions are not possible in the target domain, we can add a penalty to each reward in each sample (r, s, a, s') of the form

$$\log p_{\text{target}}(s'|s, a) - \log p_{\text{source}}(s'|s, a) = \log p(s'|s, a, \text{target}) - \log p(s'|s, a, \text{source})$$

This penalty essentially gives higher reward for transitions that are likely in the target domain but unlikely in the source domain. This basically encourages exploration in the target domain (but the actions we are exploring may not always be good). This term also penalizes transitions that can happen in the source domain but not in the target domain (e.g. if there some wall blocking a path in the target domain). To get this penalty, rather than training two dynamics models, another way is to train two discriminators that output $p(\text{target}|s, a, s')$ and $p(\text{target}|s, a)$ we can then use the output of the discriminators to form the expression above

$$\begin{aligned}
\log p(s'|s, a, \text{target}) - \log p(s'|s, a, \text{source}) &= \log \frac{p(s', s, a, \text{target})}{p(s, a, \text{target})} - \log \frac{p(s', s, a, \text{source})}{p(s, a, \text{source})} \\
&= \log \frac{p(\text{target}|s', s, a)p(s', s, a)}{p(\text{target}|s, a)p(s, a)} - \log \frac{p(\text{source}|s', s, a)p(s', s, a)}{p(\text{source}|s, a)p(s, a)} \\
&= \log \frac{p(\text{target}|s', s, a)}{p(\text{target}|s, a)} + \log \frac{p(s', s, a)}{p(s, a)} - \log \frac{p(\text{source}|s', s, a)}{p(\text{source}|s, a)} - \log \frac{p(s', s, a)}{p(s, a)} \\
&= \log p(\text{target}|s', s, a) - \log p(\text{source}|s', s, a) + \log p(\text{source}|s, a) - \log p(\text{target}|s, a)
\end{aligned}$$

20.3 Fine-Tuning

Pretraining in RL generally does not work as well as in supervised learning where the tasks are very diverse. For instance, when training image classifiers and LLMs, there are many available datasets of examples, but with RL, the datasets are generally for specific tasks that may not produce generalizable feature representations. Moreover, pretraining a policy causes the policy to be more deterministic at convergence (e.g. policy will be less random and commit to actions with higher reward in source domain), which can limit exploration in the target domain during fine-tuning.

21 Meta-learning

With meta-learning, the problem we are trying to solve is how to perform a task well given only little amounts of data for that task but have a lot of data for other tasks. Using the other datasets, we want a model or algorithm that let's the model learn how to learn, so that it can learn a new task quickly with smaller amounts of data. In normal supervised learning, we are given a dataset $D = \{(x^k, y^k) : k \in \{1, \dots, K\}\}$ where each sample (x, y) is an input and output pair. We then try to find a function parameterized by θ f_θ that minimizes

$$\sum_{k=1}^K L(f_\theta(x^k), y^k)$$

In meta-learning, we are given a dataset of pairs of datasets $D_{\text{dataset}} = \{(D_{\text{train}}^k, D_{\text{test}}^k) : k \in \{1, \dots, K\}\}$ where each $(D_{\text{train}}, D_{\text{test}})$ pair are separate datasets with $D_{\text{train}}^k = \{(x_r^{k,m}, y_r^{k,m}) : i \in \{1, \dots, m\}\}$ and $D_{\text{test}}^k = \{(x_s^{k,n}, y_s^{k,n}) : n \in \{1, \dots, N\}\}$. The goal of meta learning is then to find

$$\sum_{k=1}^K \sum_{n=1}^N L(f_\theta(D_{\text{train}}^k, x_s^{k,n}), y_s^{k,n})$$

At test time, we can think of input to the meta-learning is the concatenation of of a training dataset D_{train} and and input x . Alternatively, we can think of meta-learning as normal supervised learning consisting of one large dataset

$$D = \{((D_{\text{train}}^k, x_s^{k,n}), y_s^{k,n}) : (k, n) \in \{1, \dots, K\} \times \{1, \dots, N\}\}$$

Another way to think about it is that we are training a model or algorithm that takes in D_{train} and outputs parameters for the specific task D_{train} corresponds to that performs well on the corresponding D_{test} for any given D_{train} .

21.1 Black Box Learning

One way to make predictions for each $(x^{k,n}, y^{k,n})$ in test set D_{test}^k is to embed the entire D_{train}^k as $e_\phi(D_{\text{train}}^k)$ using a sequence model and use the embedding plus $x_s^{k,n}$ to predict $y_s^{k,n}$ with a separate network n_ψ

$$f_\theta(D_{\text{train}}^k, x_s^{k,n}) = n_\psi(e_\phi(D_{\text{train}}^k), x_s^{k,n})$$

where $\theta = (\phi, \psi)$. This is similar to multi-task or goal conditioned learning, but the context of the task is learned and inferred through $e_\phi(D_{\text{train}}^k), x_s^{k,n})$ rather than given

21.2 Feature Matching

Another way to make predictions for test points in D_{test}^k is to use feature embeddings for individual test points, where we have some embedding function e_ϕ for training set inputs $x_r^{k,m}$ and an embedding function g_ψ for test points $x_s^{k,n}$. We then model the probability that $x_s^{k,n}$ is most similar to $x_r^{k,m}$, $S^{k,m,n}$ as proportional to the dot product of their embedding vectors

$$p(S^{k,m,n}) = \frac{\exp(e_\phi(x_r^{k,m})^T g_\psi(x_s^{k,n}))}{\sum_{m=1}^M \exp(e_\phi(x_r^{k,m})^T g_\psi(x_s^{k,n}))}$$

and model the probability that

$$p(y_s^{k,n} = y) = \sum_{m: y^{k,m} = y} p(S^{k,m,n})$$

An alternative is to take an average of the feature embeddings from the target class before taking the exponent, such that

$$p(y_s^{k,n} = y^o) = \frac{\exp((c_y^k)^T g_\psi(x_s^{k,n}))}{\sum_{o=1}^{|Y|} \exp((c_y^o)^T g_\psi(x_s^{k,n}))}$$

where

$$c_y^k = \frac{1}{|\{m : y^{k,m} = y\}|} \sum_{\{m: y^{k,m} = y\}} e_\phi(x_r^{k,m})$$

Here $\theta = (\phi, \psi)$

21.3 Model Agnostic Meta Learning

Another way we can implement meta learning is to just have a normal model f_θ that takes an input x and outputs y and train over D_{train}^k as normal except take gradient steps that position θ such that taking gradient steps on D_{train}^k would lead to parameters θ that perform well on both D_{test}^k and D_{train}^k . Essentially, we are trying to find an initialization of θ that tends to align the gradient between D_{test}^k and D_{train}^k for all possible (D_{test}^k and D_{train}^k) in the training distribution of pairs of datasets. To do so, we try to optimize the following objective

$$L_{\text{Meta}} = \sum_{k=1}^K \sum_{m=1}^M L(f_{s(\theta, D_{\text{train}}^k)}(x^{k,m}), y^{k,m})$$

where

$$s(\theta, D_{\text{train}}^k) = \theta - \alpha \nabla_\theta \sum_{m=1}^M L(f_\theta(x^{k,m}), y^{k,m})$$

which is basically the result of running one training step on D_{train}^k . We can also (which is done in practice) run the training step on D_{train}^k for multiple steps.

This essentially lets the outer ∇_θ minimize the loss on the testing set for $s(\theta, D_{\text{train}}^k)$, meaning performing gradient descent on the outer ∇_θ is essentially shifts θ to where training on D_{train}^k tends to improve D_{test}^k . Essentially, this makes it so that fine tuning starting at θ at the end of the training procedure for any D_{train} we get at test time produces the best results for D_{test} . At test time, personal intuition says we should run fine tuning on D_{train} for the same number of steps of the inner optimization over D_{train}^k , which we denote as I , during training since we aligned the gradients for D_{train}^k and D_{test}^k for θ after I of fine tuning. However, in practice, we often see better results fine tuning for more steps at test time than I .

Note that this method is basically pretraining but positioning the initial θ such that I steps of fine tuning produces good results on any task from the distribution. This is different from the previous two methods such as blackbox learning which do not perform further training on the test set that shift model parameters (since the RNN forward pass on the D_{train} is the learning in that case). Similarly, we are assuming we have learned sufficiently good representations in feature matching. Note that this be used with policy gradients but can be difficult to implement for actor critic methods since actor critic methods are not strict gradient descent but are a form of TD learning (e.g. TD(0)).

21.4 Meta-Learning in RL

In both supervised meta-learning and meta-learning in RL, our training and test (this test references meta-testing dataset and not the D_{test}^k we use for training) dataset both contain sample of tasks. In supervised meta-learning each task is represented as a data distribution, which is represented by samples from that distribution. For instance, each sample from a supervised meta learning dataset ($D_{\text{train}}^k, D_{\text{test}}^k$) can be thought of as sampling a distribution $p^k(x, y)$ from a distribution of distributions $p(p(x, y))$ and we sample $p^k(x, y)$ to get some datapoints, which we split into ($D_{\text{train}}^k, D_{\text{test}}^k$) that we use to represent the task $p^k(x, y)$. In meta-RL, each task is an MDP, so each sample from the meta-RL dataset is an MDP which we can interact with to gather samples to represent that MDP.

21.5 Black Box Meta-RL

Similar to black box supervised meta learning, we have some RNN that is used to encode the task. In supervised meta-learning, we encode the concatenation of samples from the distribution, but in meta-RL, we encode the concatenation of all transitions from all encountered trajectories (e.g. a meta-trajectory). For instance, in N-shot learning, we are allowed N rollouts per task, $(\tau_1, \tau_2, \dots, \tau_n)$ where $\tau_i = ((s_1^i, a_1^i, s_2^i, r_1^i), (s_2^i, a_2^i, s_3^i, r_2^i), \dots, (s_T^i, a_T^i, s_{T+1}^i, r_T^i))$. In the timestep t of rollout i rollout the RNN encodes all previous transitions (including those from previous rollouts)

$$\begin{aligned} & ((s_1^1, a_1^1, s_2^1, r_1^1), (s_2^1, a_2^1, s_3^1, r_2^1), \dots, (s_T^1, a_T^1, s_{T+1}^1, r_T^1), (s_1^2, a_1^2, s_2^2, r_1^2), (s_2^2, a_2^2, s_3^2, r_2^2), \\ & \dots, (s_T^2, a_T^2, s_{T+1}^2, r_T^2), \dots, (s_1^i, a_1^i, s_2^i, r_1^i), (s_2^i, a_2^i, s_3^i, r_2^i), \dots, (s_3^{t-1}, a_3^{t-1}, s_4^{t-1}, r_3^{t-1})) \\ & = (\tau_1, \dots, \tau_{i-1}, (s_1^i, a_1^i, s_2^i, r_1^i), (s_2^i, a_2^i, s_3^i, r_2^i), \dots, (s_{t-1}^i, a_{t-1}^i, s_t^i, r_{t-1}^i)) \end{aligned}$$

into some context embedding h and the policy takes in h, s_t to produce a_t . Just as before, we also do not need to do any training at meta-testing time and only need to rollout the policy N times and use that as the context for the task.

This method automatically learns how to get the best policy and how to explore. As a small example, let's say we have a distribution of MDPs with one state and 4 actions and a reward of 1 assigned randomly to one of the four actions. In each training rollout, the contextual policy will learn that some directions give no rewards, but once a reward is found for a particular action by doing more rollouts for that MDP, the policy will learn to repeat the action with the reward. This will also work at test time where the reward is for a different action as long as the contextual policy learns we should repeat the action for which we see the reward and the policy sees a reward at test time. Note that just like black box supervised meta-learning, we do typically do not change the model parameters after training and just let the sequence model forward pass do the learning. This can also be viewed as solving a POMDP where the true state $\tilde{s}$ is the concatenation of (s, z) but we are only able to observe s . And one way to learn the context is to use the ELBO loss

$$\mathbb{E}_{z \sim q_\phi(z|c^{i,t})}[R^{i,t}(z) + \beta D_{KL}(q_\phi(z|c^{i,t})||p(z))]$$

where $c^{i,t} = (\tau_1, \dots, \tau_{i-1}, (s_1^i, a_1^i, s_2^i, r_1^i), (s_2^i, a_2^i, s_3^i, r_2^i), \dots, (s_{t-1}^i, a_{t-1}^i, s_t^i, r_{t-1}^i))$, where $R^{i,t}(z)$ is the negative of the something like a critic loss (e.g. $(Q_\theta(s, a, z) - (r + V(s', \text{stopgradient}(z))))^2$), essentially optimizing ϕ to produce a z that closely matches the rewards seen for the current environment.